“TRUEDATA”

A PROJECT REPORT

Submitted by

BHATT TIRTHRAJ (ET24MTCA005)

Under the Guidance of

PROF. BIKASH PRADHAN

In fulfilment for the award of the degree

Of

Master of Computer Applications

At

Sarvajanik College of Engineering & Technology, Surat

Sarvajanik University, Surat

MAY, 2026

Sarvajanik University

Sarvajanik College of Engineering and Technology, Surat.

Department of Computer Application

Academic Year 2025-26

Date: 02/05/2026

CERTIFICATE

This is to certify that the project entitled "TrueData" has been submitted by Bhatt Tirthraj (ET24MTCA005) towards fulfillment of the degree of Master of Computer Applications (MCA) in 4th Semester of Sarvajanik University, Surat during the academic year 2025-26.

Guide Name: Prof. Bikash Pradhan

(Guide's Signature)

Dr. Gayatri Kapadia
Head, Department of Computer Application

Examiner's Signature:

1. _____

2. _____

3. _____

Company Certificate

No Code Certificate

Self-Declaration

Title of the project: TrueData

Enrollment No	Student Name
ET24MTCA005	Bhatt Tirthraj

I hereby declare that the above-mentioned project report submitted by me and has been prepared by me and is original in its content and it has not been submitted anywhere else.

I confirm that the report is only prepared for academic requirement, not for any other purpose. It might not be used by anyone for any other purpose.

Student Signature:

_____

Acknowledgement

I extend my deepest gratitude to **Mr. Kapil Marwaha** and **Mr. Utkarsh Kapadia**, founders and CEOs of **TrueData**, for their invaluable support and guidance throughout my project. Their vision, sincerity, and motivation have been constant sources of inspiration for me.

I am also immensely grateful to my **Project Mentor,Prof Bikash Pradhan** and our **Head of Department, Dr. Gayatri Kapadia ,** for providing me with the opportunity to undertake this project and for their unwavering guidance. Their methodological approach and clear instructions have been instrumental in the success of my project. Working under their mentorship has been a privilege and an honor.

Furthermore, I express my heartfelt appreciation to Sarvajanik College of Engineering and Technology, Surat, for granting me the opportunity to work on a real-time project. The support and guidance extended by all the teaching staff of the Masters of Computer Application Department have been instrumental in successfully completing my project. I would also like to extend my sincere regards to all the non-teaching staff of the Masters of Computer Application Department for their timely support and assistance.

Once again, thank you all for your friendship, empathy, and sense of humor, which have made this journey even more memorable.

Yours Sincerely,
Bhatt Tirthraj (ET24MTCA005)

INDEX

1. Introduction

1.1 Existing System

Prior to the deployment of this modernized platform, the existing infrastructure exhibited several critical limitations:

- Absence of a structured blog interaction system, limiting organic SEO growth and user engagement.
- Lack of a verified testimonials framework, making it difficult to establish corporate trust.
- No automated chatbot support, requiring manual intervention for repetitive user inquiries.
- Heavy reliance on manual extraction of complex financial data from dense RHP PDFs, which is error-prone and time-consuming.
- High vulnerability to spam submissions on public-facing forms.
- Absence of a dedicated C# library enhancement with corporate and mutual fund endpoints, resulting in inefficient, scattered API calls and bloated controller logic.
- No test page for checking of greek api.

1.2 Need for the New System

The proposed system aims to automate complex financial data workflows and streamline client communications while ensuring enterprise-grade security and scalability. Key requirements include:

- A seamless registration and secure login process utilizing session-based authentication.
- Advanced AI-driven extraction of complex financial data (RHP PDFs), automatically translating unstructured tables into clean, relational database records.
- A fully integrated, real-time WhatsApp Chatbot architecture to provide instant, 24/7 automated support and query resolution.
- Comprehensive Profile, Blog, and Testimonial management capabilities equipped with robust administrative moderation.
- Enhanced data security and strict anti-spam protection, utilizing Google reCAPTCHA v3 and Honeypot methodologies.

- Integration of a dedicated C# Class Library acting as middleware to efficiently fetch and serve real-time Corporate Actions and Mutual Fund (NAV) endpoints.
- Live WebSocket integrations for the zero-latency broadcasting of NSE corporate announcements and market updates directly to users.
- A functional with multiple selected option for the checking greeks api .

1.3 Objective of the New System

The primary objectives of the proposed system are:

- To develop a robust, user-friendly financial data platform equipped with an SEO-optimized blog and an administratively verified testimonial module.
- To ensure a highly secure authentication and interaction system utilizing session-based login, Google reCAPTCHA v3, and Honeypot methodologies to prevent automated spam and SQL injection.
- To automate the complex extraction of unstructured financial data from Red Herring Prospectus (RHP) PDFs into a structured, relational database using AI-driven pipelines.
- To facilitate instant, 24/7 client engagement and automated query resolution through a fully integrated, state-machine driven WhatsApp chatbot architecture.
- To provide comprehensive profile and subscription management, allowing users to seamlessly track their data queries and manage their system access.
- To decouple external data fetching by implementing a dedicated C# Class Library that acts as middleware to serve real-time corporate actions and mutual fund (NAV) metrics.
- To deliver critical market updates with zero latency via WebSocket integrations that broadcast live NSE corporate announcements directly to users.
- To easily check data of the greek api .

1.4 Problem Definition:

Existing financial data applications and operational workflows often face challenges related to:

- Current operational systems fail to handle structured blog interactions effectively or provide a platform for verified reviews.
- High susceptibility to spam and malicious automated submissions on public-facing forms.
- Existing solutions completely fail to extract financial data automatically. They struggle to process varied document layouts, execute necessary unit conversions (Millions, Crores, Lakhs), and parse multi-line headers required for accurately extracting Balance Sheets, Profit & Loss statements, and Cash Flow metrics.
- Inefficient and scattered external API data fetching, causing bloated controller logic and delayed market updates.
- Lack of automated customer support, requiring constant manual intervention for repetitive user queries.

The proposed system addresses these issues by:

- Providing a structured, SEO-friendly blog management system and an administratively verified testimonial platform.
- Implementing robust anti-spam security mechanisms, including Google reCAPTCHA v3 and hidden Honeypot fields, to protect public interactions.
- Automating complex RHP PDF data extraction using an advanced AI pipeline to seamlessly map, convert, and store tabular financial data into a structured relational database.
- Developing a dedicated C# Class Library to efficiently fetch, decouple, and serve real-time Corporate Actions and Mutual Fund (NAV) endpoints.

1.5 Core Component:

1. Blog Management System

- **Objective:** Provide a structured, SEO-friendly platform for publishing financial articles to maximize organic search visibility and user engagement.
- **Features:**
  - **Content Creation & Editing:** Admins can author, format (HTML), and publish financial blogs.
  - **SEO Optimization:** Automated generation of canonical tags, dynamic meta descriptions, and custom URL slugs.
  - **Search & Filtering:** Users can browse articles, filter by category, and search using specific keywords.
  - **Interactive Comments:** Users can post comments on blogs, seamlessly protected by backend anti-spam mechanisms.

2. Testimonial Module

- **Objective:** Establish corporate trust by collecting and displaying authentic, admin-verified user reviews.
- **Features:**
  - **Secure Submission Form:** Users can submit ratings, detailed reviews, and profile pictures.
  - **Verification Gate:** Admins must explicitly approve and verify testimonials before they are published on the public feed.
  - **Homepage Feature Control:** Ability for admins to flag specific high-quality reviews to be featured prominently on the main landing page.

3. Admin Panel

- **Objective:** Serve as the centralized hub for system moderation, user management, and overarching platform controls.
- **Features:**
  - **Content Moderation:** Intuitive interface to approve, reject, or delete blog comments and user testimonials.
  - **System Auditing:** Monitor chatbot interaction logs, system errors, and user queries in real-time.

- **Extraction Management:** Interface for administrators to upload complex RHP PDFs and trigger the automated data extraction pipeline.

4. Chatbot System

❖ **Objective:** Provide instant, 24/7 automated support and query resolution to clients without requiring manual human intervention.
❖ **Features:**

- **Real-time WhatsApp Integration:** Fully integrated Webhook architecture to seamlessly receive and send WhatsApp messages.
- **State-Machine Logic:** Track conversation steps (e.g., 'NEW', 'PENDING') to handle multi-part financial queries.
- **Session Logging:** Link user phone numbers to specific chat sessions and log all interactions securely in the database (`ChatLogs` and `UserQueries`).

5. RHP Data Extraction Module

❖ **Objective:** Automate the complex translation of unstructured financial PDF documents into clean, structured, and relational database records.
❖ **Features:**

- **Advanced PDF Parsing:** Utilize Python-based AI parsing (PdfPig) to read complex graphical tables.
- **Unstructured-to-Structured Mapping:** Map raw Profit & Loss, Balance Sheet, and Cash Flow tables into standard accounting database fields.
- **Dynamic Unit Conversion:** Automatically handle complex mathematical unit conversions (e.g., Millions to Crores, Lakhs) securely within the backend logic.

6. C# API Integration Library & Live Notifications

❖ **Objective:** Decouple external data fetching and deliver critical market updates with zero latency.
❖ **Features:**

- **Corporate & Mutual Fund Endpoints:** Dedicated C# middleware to securely fetch and serve real-time corporate actions (dividends, splits) and mutual fund metrics (NAV).

- **WebSocket Broadcasts:** Real-time integrations broadcasting live NSE corporate announcements directly to connected clients.

7. Anti-Spam & Security Framework

❖ **Objective:** Protect public-facing forms from automated bot attacks, SQL injection, and malicious spam without degrading the user experience.

❖ **Features:**

- **Google reCAPTCHA v3:** Invisible, score-based risk assessment to validate genuine users.
- **Honeypot Strategy:** Hidden fields designed to trap and silently discard automated bot traffic before it hits the database.

1.6 Project Profile

- **Project Name:** TrueData Web Application
- **Technology Stack:** ASP.NET MVC, Microsoft SQL Server, Python, Pymssql, C#
- **Domain:** Financial Data Analytics & Web Application
- **Type:** Product-based Web Application

Project Overview The project is a robust, centralized financial data platform that enables administrators to manage SEO-optimized blogs and verified testimonials, while providing users with instant, automated support via a WhatsApp chatbot. Unlike many existing platforms, this system prioritizes high-fidelity financial data accessibility by automating the extraction of complex Red Herring Prospectus (RHP) PDFs. It also incorporates advanced features like zero-latency WebSocket broadcasting for NSE corporate announcements and a dedicated C# Class Library for real-time Corporate and Mutual Fund data fetching.

Key Features

- **Secure Authentication & Anti-Spam** – Session-based login with Google reCAPTCHA v3 and Honeypot methodologies protecting public forms.
- **SEO-Optimized Blog Management** – Admin tools for content creation, canonical tag generation, and dynamic meta descriptions.
- **Verified Testimonial Module** – Secure collection, admin verification, and homepage display of user reviews.
- **Automated RHP Data Extraction** – Python-based (PdfPig) parsing of complex financial PDFs into structured database records with dynamic unit conversions.
- **Real-time WhatsApp Chatbot** – State-machine driven automated support and query tracking.
- **Live Market Notifications** – Zero-latency WebSocket broadcasting of NSE corporate announcements.
- **C# API Integration Library** – Dedicated middleware for fetching real-time Corporate Actions and Mutual Fund (NAV) endpoints.

Tech Stack

Category	Technologies	Purpose / Description
Frontend	HTML5, CSS3, Bootstrap 5, JavaScript, jQuery, Razor View Engine	Provides a responsive, SEO-friendly user interface.
Backend Web Framework	ASP.NET MVC 5, .NET Framework 4.8 (C#)	Handles core application logic, routing, and user session management.
Backend Data Services	Python, Pymssql	Handles advanced AI-driven PDF extraction and data structuring.
Database	Microsoft SQL Server 2022 (Stored Procedures + ADO.NET / Dapper)	Securely stores user profiles, blog content, chat logs, and parsed financial data.
Integration Middleware	Custom C# Class Library	Decouples and manages external API requests for market data.

Project Benefits

- **Automated Data Processing:** Eliminates error-prone manual extraction of financial data through advanced Python PDF parsing.
- **Enhanced Security:** Protects user data and prevents spam attacks using robust validation and invisible reCAPTCHA.
- **Instant Client Support:** Real-time query resolution and interaction tracking via the integrated WhatsApp chatbot.
- **High Performance & Scalability:** Decoupled C# middleware and optimized SQL procedures ensure efficient handling of heavy data loads.
- **Improved Digital Footprint:** Deeply integrated SEO workflows maximize organic visibility for financial content.

1.7 Assumptions and Constraints

Assumptions

- **Internet Connectivity:** It is assumed that all end-users and administrators will have stable internet access to interact with the TrueData Web Application, submit queries, and access extracted data.
- **Administrative Management:** It is assumed that administrators will actively and regularly manage content, including reviewing blog posts, verifying testimonials, and monitoring system logs.
- **Document Structure:** The system assumes that the target financial prospectuses (RHPs) will generally adhere to recognizable and standard tabular formats, allowing the AI parsing engine to identify and extract relevant financial data effectively.

Constraints

- **PDF Format Dependency:** The accuracy and efficiency of the automated RHP extraction process are highly dependent on the structural integrity of the complex PDF formats provided. Highly irregular or poorly scanned documents may result in extraction failures or inaccuracies.
- **Server Performance Limitations:** Executing highly complex PDF extraction and AI parsing tasks, especially concurrently, may impose limitations on server performance and processing speed.
- **API Integration Limitations:** The delivery of chatbot messages and interactions is constrained by the limitations and rate limits of third-party API integrations (e.g., WhatsApp API).

1.8Advantages and Limitations of the Proposed System

Advantages:

1. **Automated Financial Data Extraction** – Highly accurate and efficient data extraction driven by a robust Python backend (PdfPig), which eliminates error-prone manual data entry by automatically structuring tabular RHP data.
2. **Robust Security Architecture** – Secure system infrastructure utilizing Google reCAPTCHA v3 and Honeypot methodologies, effectively protecting public-facing forms from automated bot attacks and malicious spam.
3. **Seamless User Experience** – Intuitive, user-friendly interface designed for rapid navigation, making it exceptionally easy for users to browse financial blogs, submit testimonials, and interact with the platform.
4. **Advanced SEO Optimization** – Deeply integrated SEO workflows, including automated canonical tags and dynamic meta descriptions, which significantly maximize organic search engine visibility.
5. **Real-Time Client Support** – A fully integrated, state-machine driven WhatsApp chatbot architecture provides instant, 24/7 automated query resolution and interaction tracking without requiring manual human intervention.
6. **High-Performance API Middleware** – A dedicated C# Class Library efficiently decouples, fetches, and serves real-time Corporate Actions and Mutual Fund (NAV) data, ensuring fast response times and clean controller logic.
7. **Zero-Latency Market Notifications** – Real-time WebSocket integrations broadcast live NSE corporate announcements and stock updates instantly to connected clients.

Limitations:

1. **Dependency on Document Formatting** – Highly irregular, non-standard, or poorly scanned PDF formats may cause the automated extraction pipeline to fail, as the AI relies on recognizable tabular structures.
2. **Continuous Maintenance Requirements** – The system requires periodic maintenance and updates, specifically for training the chatbot's conversational NLP models and updating the PDF extraction mapping rules as document standards change.
3. **High Dependency on Third-Party APIs** – Heavy reliance on external data providers (NSE/AMFI endpoints) and third-party communication channels (WhatsApp Webhook API) may introduce risks if rate limits are exceeded, pricing models change, or external servers experience downtime.
4. **Server Processing Overhead** – Executing complex AI-based PDF parsing and mathematical unit conversions requires significant server resources, potentially leading to performance bottlenecks during high-concurrency document uploads.

5. **Limited Offline Functionality** – Core features such as live market WebSocket broadcasts, chatbot interactions, and real-time external data fetching rely entirely on a stable internet connection, limiting offline usability.

2 Requirement Determination & Analysis

2.1 Requirement Determination

2.1.2 Non-Functional Requirements

1. **User & Admin Actions on Application**

❖ **Account & Profile Management**

- **Login & Authentication:** Secure, session-based login mechanism for administrators to access the centralized control panel.
- **Profile & Query Management:** System tracks user phone numbers securely, allowing users to view their chatbot interaction history and data queries.

2. **Content Management (Web CMS)**

❖ **Blog Management**

- **Blog CRUD Operations:** Administrators can Create, Read, Update, and Delete SEO-friendly financial articles.
- **Comment & Reply System:** Users can seamlessly engage with blog posts through hierarchical comments and replies.

3. **Testimonial System**

- **Testimonial Submission:** Users can submit reviews, ratings, and profile pictures via a public form.
- **Verification Gate:** Administrators must explicitly review, verify, and approve testimonials before they are published to the live environment.

4. **Automated Data Processing**

❖ **RHP PDF Extraction Pipeline**

- **Raw Table Interpretation:** The Python-based AI parser (PdfPig) accurately reads complex, unstructured tabular data from uploaded RHP PDFs.
- **Data Mapping & Conversion:** Automatically maps extracted data to standard accounting fields and executes dynamic mathematical unit conversions (e.g., Millions to Crores).
- **Secure Database Saving:** Safely inserts the cleaned, structured financial data directly into SQL Server using `pymssql`.

❖ **C# API Integration Library**

- **Corporate & Mutual Fund Endpoints:** Dedicated C# middleware dynamically fetches, parses, and serves Corporate Actions and Mutual Fund NAV data.

5. **Communication & Real-Time Interactions**

❖ **Chatbot Integration**

- **WhatsApp Query Processing:** System receives, interprets, and responds to user queries via an automated WhatsApp Webhook.
- **State Logic Management:** The chatbot accurately manages conversational states (e.g., 'NEW', 'PENDING') to handle multi-step interactions seamlessly.

❖ **Live Market Notifications**

- **WebSocket Broadcasts:** Real-time delivery of NSE corporate announcements and market updates with zero latency.

2.1.2 Non-Functional Requirements

1. **Availability & Scalability**

- **Availability:** The platform, particularly the WhatsApp Chatbot and WebSocket notification system, must be available 24/7 to provide uninterrupted automated support.
- **Scalability:** The database and extraction pipelines must be scalable to handle concurrent automated data processing tasks and high web traffic without degradation in service.

2. **Security**

- **Anti-Spam Protection:** Implementation of Google reCAPTCHA v3 and Honeypot logic on all public-facing forms (comments, testimonials) to prevent automated bot submissions.
- **Data Security & SQL Protection:** Use of parameterized queries and ORM frameworks (Dapper/ADO.NET) to provide strict SQL Injection protection and secure database interactions.

3. **Performance Optimization**

- **Environment Containment:** Utilization of Python Virtual Environment containment (`docling-env`) to isolate the RHP extraction pipeline, ensuring optimal memory management and preventing dependency conflicts.
- **Decoupled Architecture:** The custom C# Class Library offloads heavy external API requests, reducing the load on primary MVC controllers and ensuring fast web application response times.

4. **Usability & Maintainability**

- **Usability:** Ensure an intuitive, highly responsive UI/UX for the web platform, allowing users to navigate blogs and submit testimonials effortlessly.

- **Maintainability:** Adhere to strict MVC coding standards and modular architecture to simplify future updates, bug fixes, and continuous training of the AI extraction mapping rules.

2.2 Targeted Users

Unlike a community-driven social platform, the TrueData Web Application utilizes a multi-tiered Role-Based Access Control (RBAC) architecture to cater to distinct user groups. This ensures a secure, moderated, and highly efficient environment for financial data consumption and system management.

❖ **User Roles & Responsibilities**

1. Anonymous Users

- **Content Discovery** – Browse public-facing, SEO-optimized financial blogs and educational articles.
- **View Testimonials** – Access the public feed to read authentic, admin-verified user reviews to establish trust before registering.

2. Registered Users

- **Platform Engagement** – Post hierarchical comments and replies on blog articles.
- **Testimonial Submission** – Submit personal ratings, detailed reviews, and profile pictures for admin approval.
- **Chatbot Interaction** – Communicate instantly with the fully integrated WhatsApp chatbot for automated support, query resolution, and account tracking.
- **Profile Management** – Securely manage personal details, view chatbot interaction history, and track submitted queries.

3. Administrators

- **System & CMS Management** – Create, edit, publish, and delete financial blogs, categories, and tags.
- **Content Moderation** – Review, verify, and approve user-submitted testimonials, and moderate blog comments to maintain platform integrity.
- **RHP Pipeline Execution** – Upload complex Red Herring Prospectus (RHP) PDFs to securely trigger the automated Python-based data extraction module.
- **Security & Auditing** – Monitor system logs, user query statuses (`UserQueries`), chatbot sessions (`ChatLogs`), and manage overall platform health.

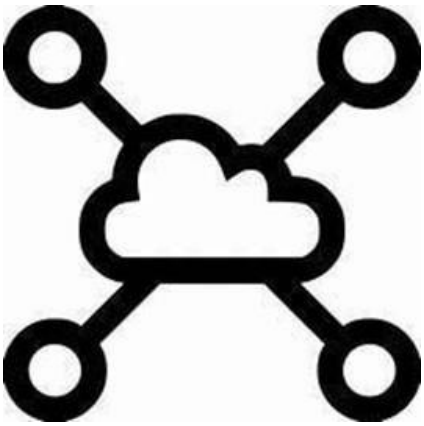
4. Financial Analysts

- **Advanced Data Utilization** – Access and utilize high-fidelity, cleanly structured Balance Sheet, P&L, and Cash Flow data extracted automatically from unstructured RHP PDFs.
- **Real-Time Market Tracking** – Consume zero-latency corporate announcements via WebSocket broadcasts and fetch live Mutual Fund NAVs leveraging the custom C# middleware.

❖ **User Experience & Benefits**

1. **Role-Specific Workflows** – Tailored interfaces and strict permission matrices ensure that users only access relevant features, maximizing both security and operational focus.
2. **Operational Efficiency** – Financial Analysts and Admins are freed from hours of error-prone manual data entry thanks to the automated RHP PDF extraction pipeline.
3. **Real-Time Engagement** – Instant automated support via the WhatsApp Webhook architecture and zero-latency market updates via WebSocket connections.
4. **Privacy & Security** – User data is securely stored in Microsoft SQL Server, with all public interaction forms heavily shielded by Google reCAPTCHA v3 and Honeypot logic to prevent spam.
5. **Seamless UI/UX** – A responsive, intuitive navigation layout built on Bootstrap 5 and Razor Pages, designed specifically to present complex financial data clearly across all devices.

3 System Design



3.1 Use Case Diagram

❖ Use Case Diagram of Admin

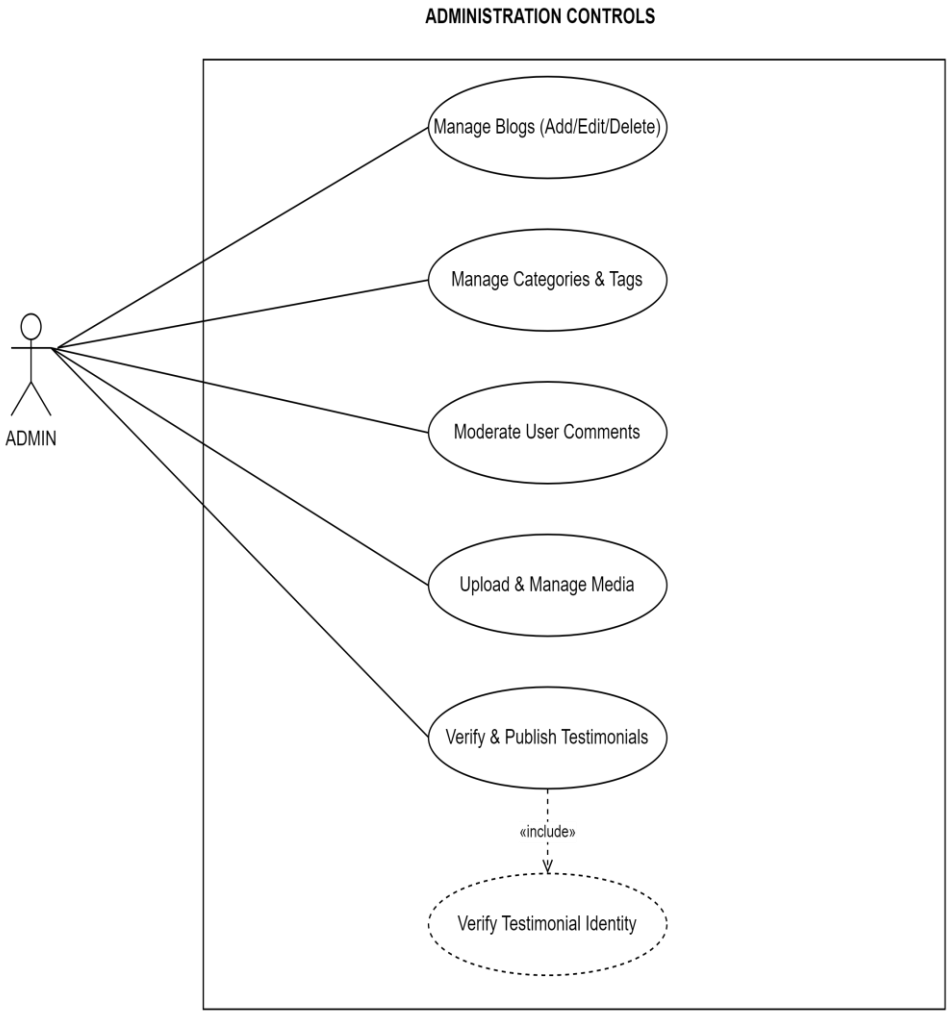


Figure 1: Use Case Diagram of Admin

❖ Use Case Diagram of User

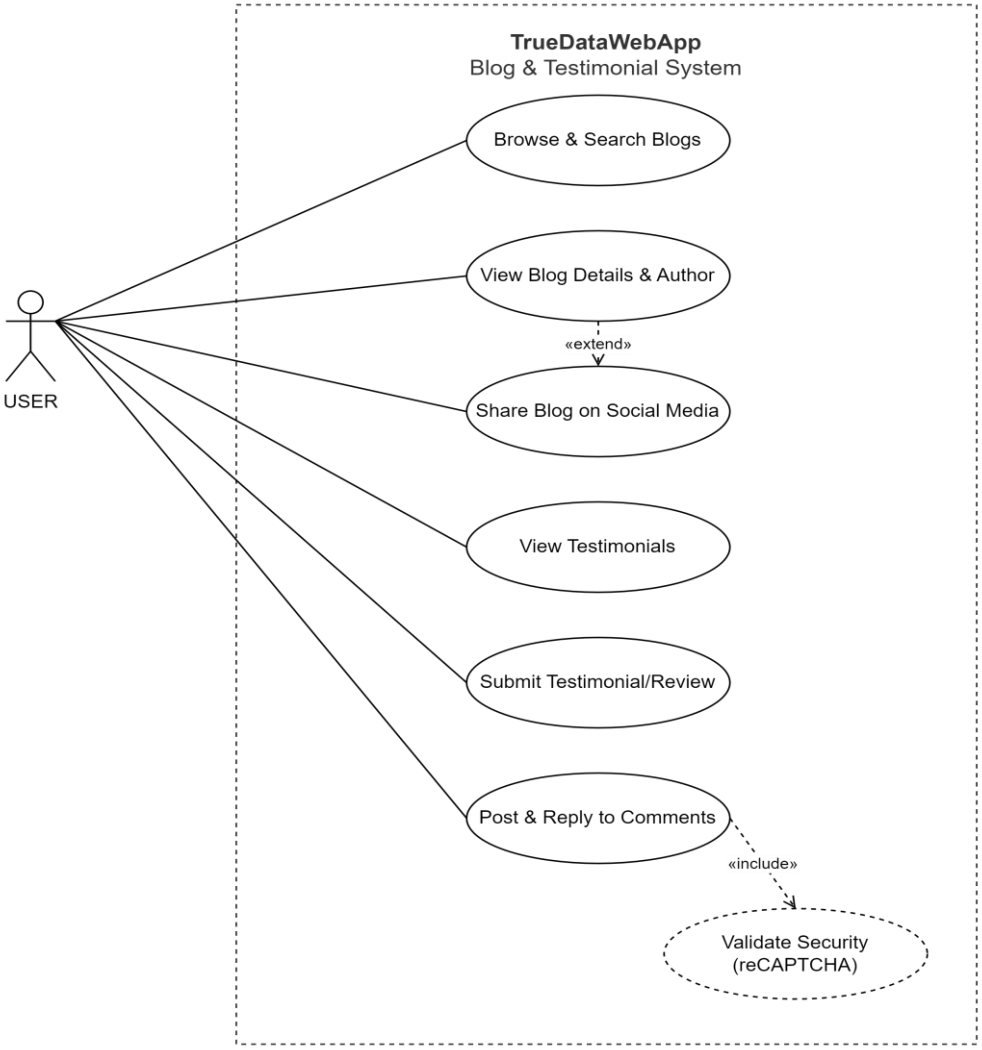


Figure 1: Use Case Diagram of User

❖ Use Case Diagram of User

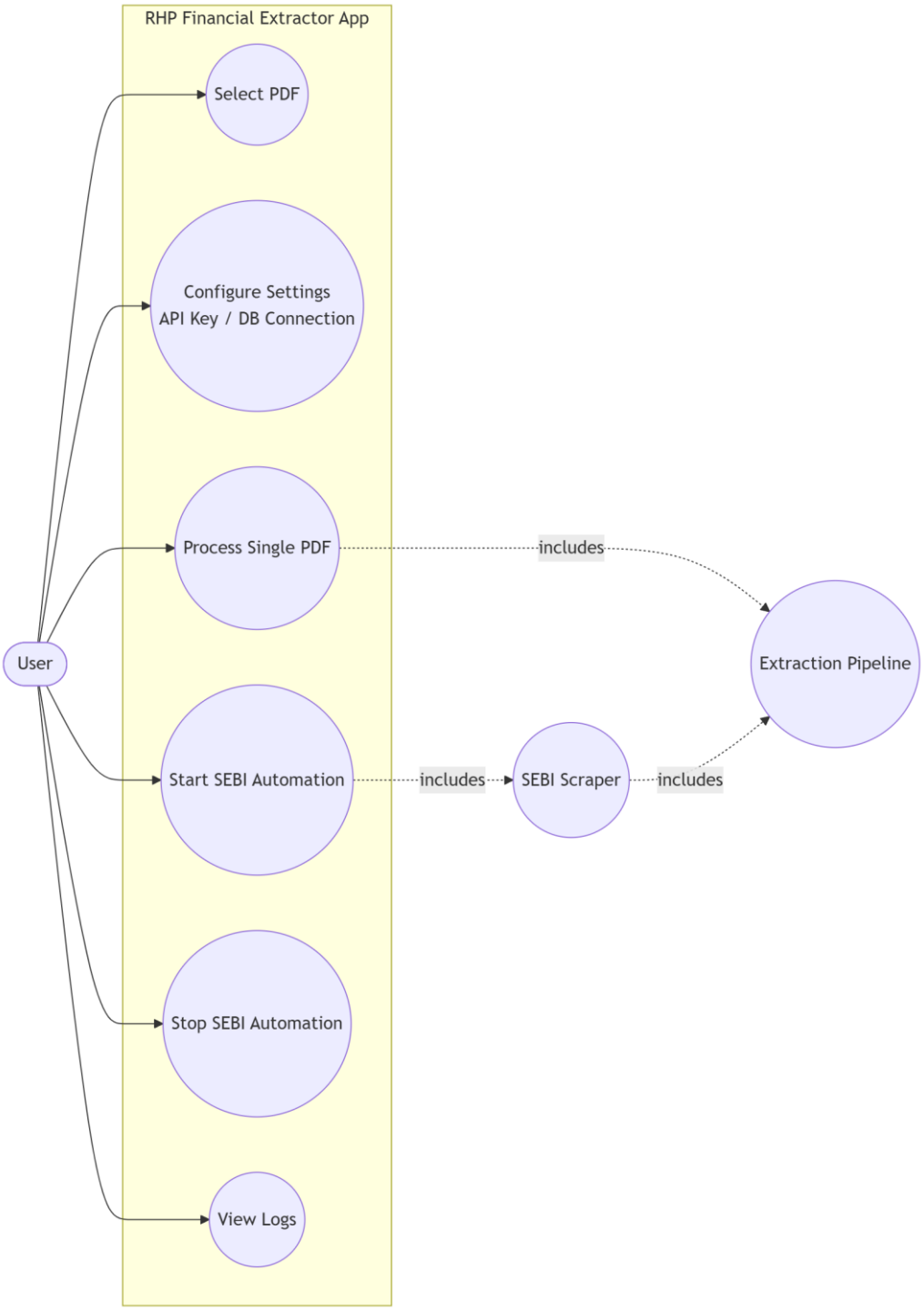


Figure 1: Use Case Diagram of User

3.2 Class Diagram

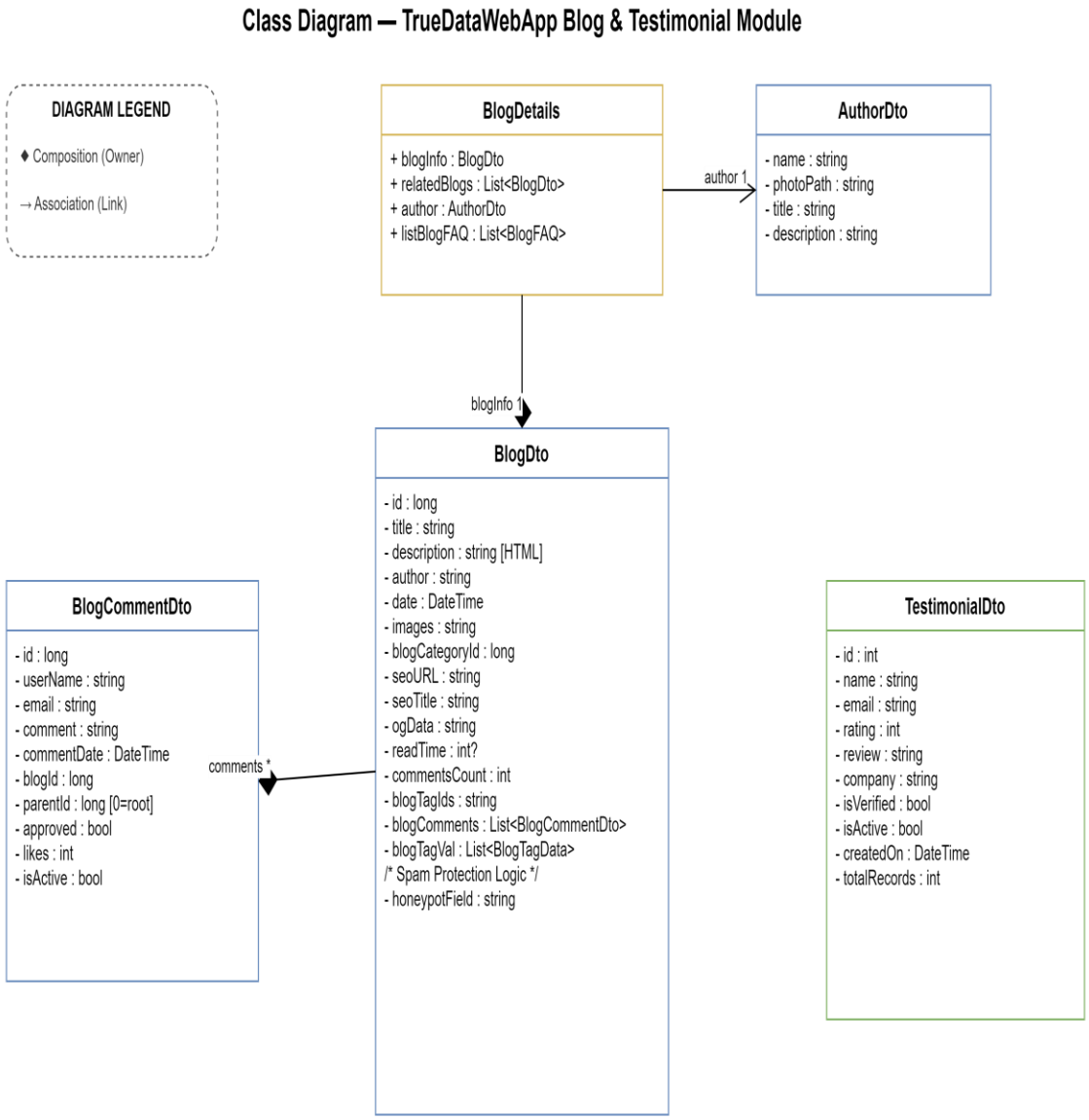


Figure 2: Class Diaram

3.3 Interaction Diagram

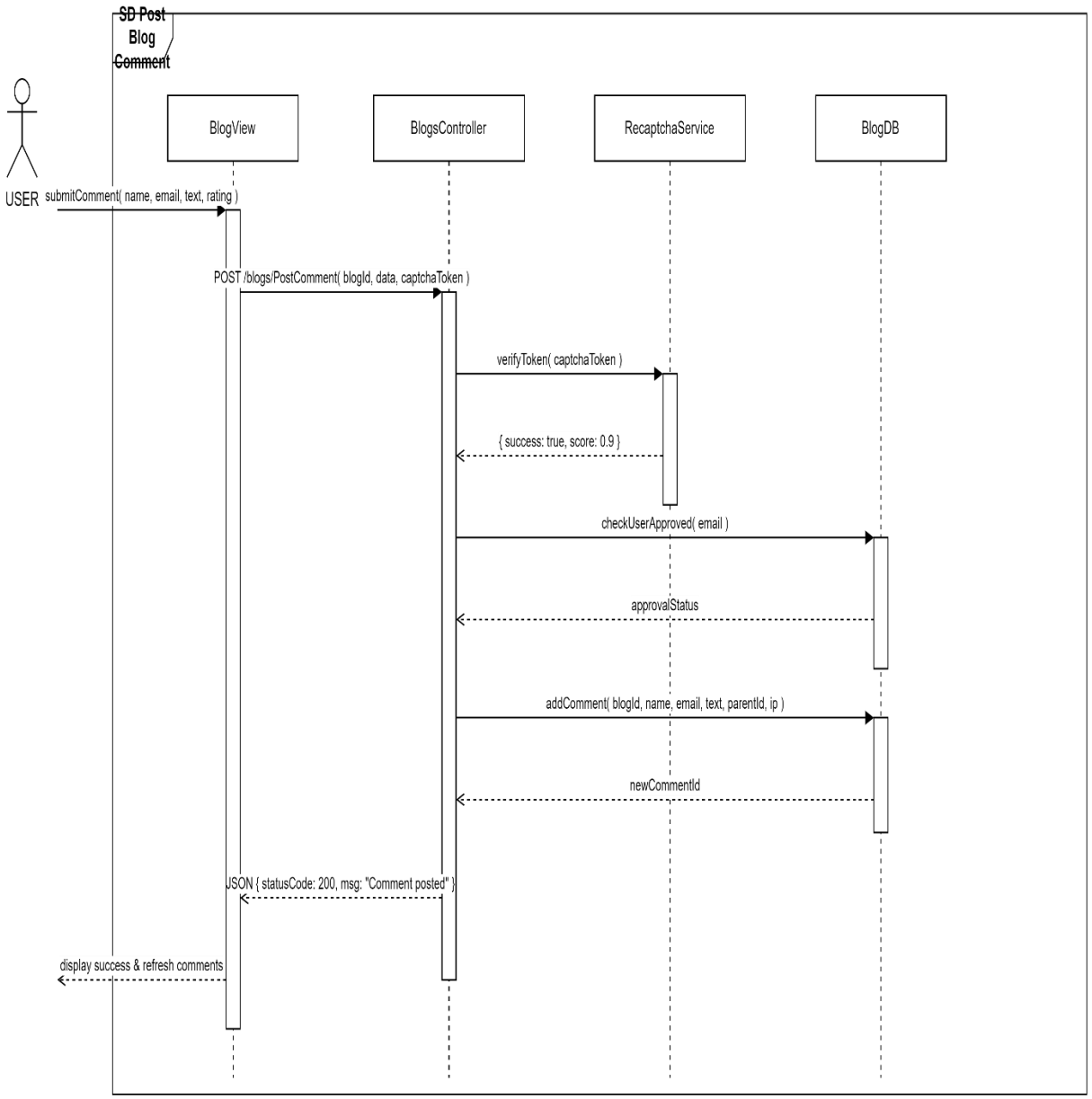


Figure 3: Interaction Diagram

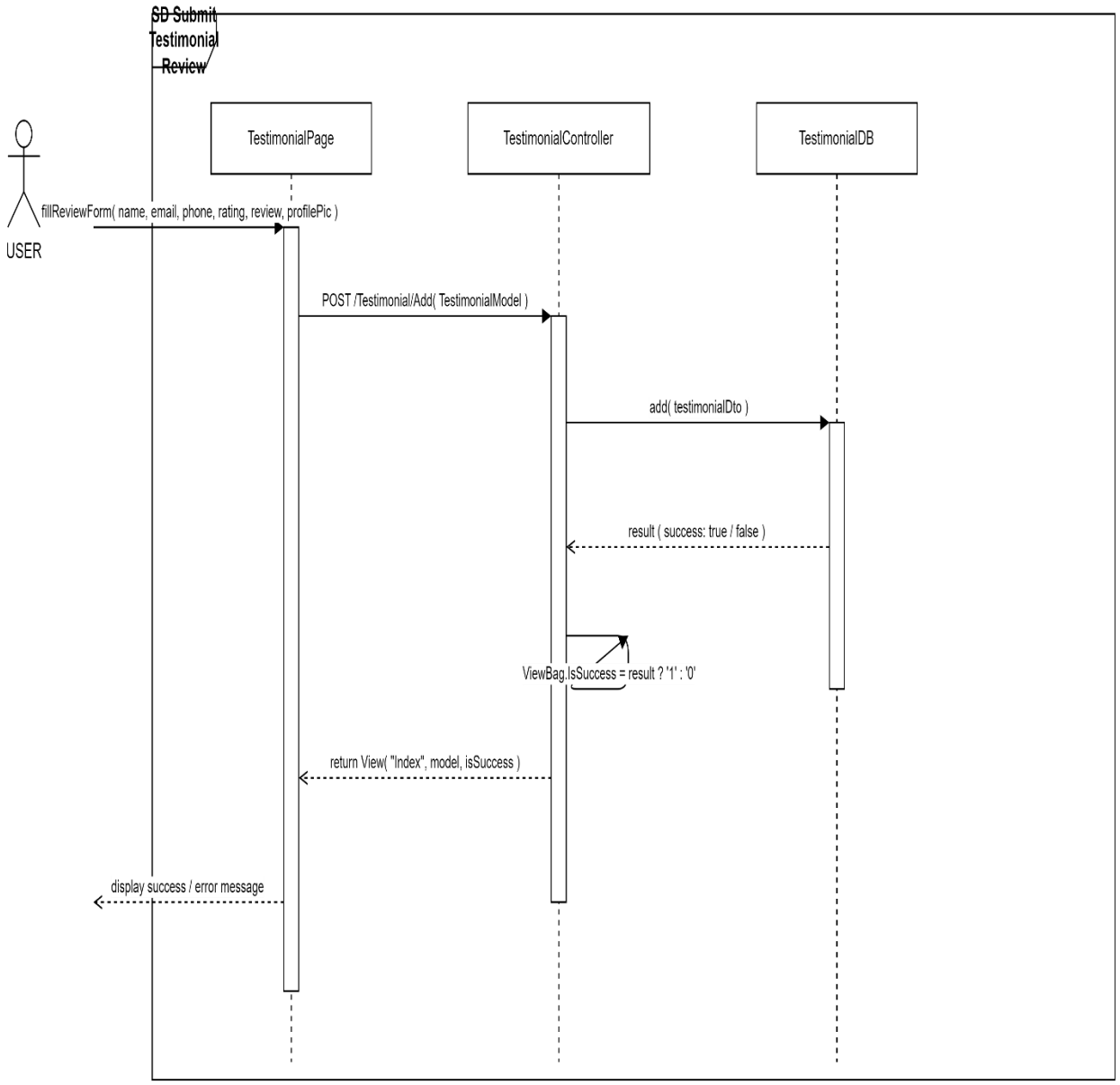


Figure 4: Interaction Diagram

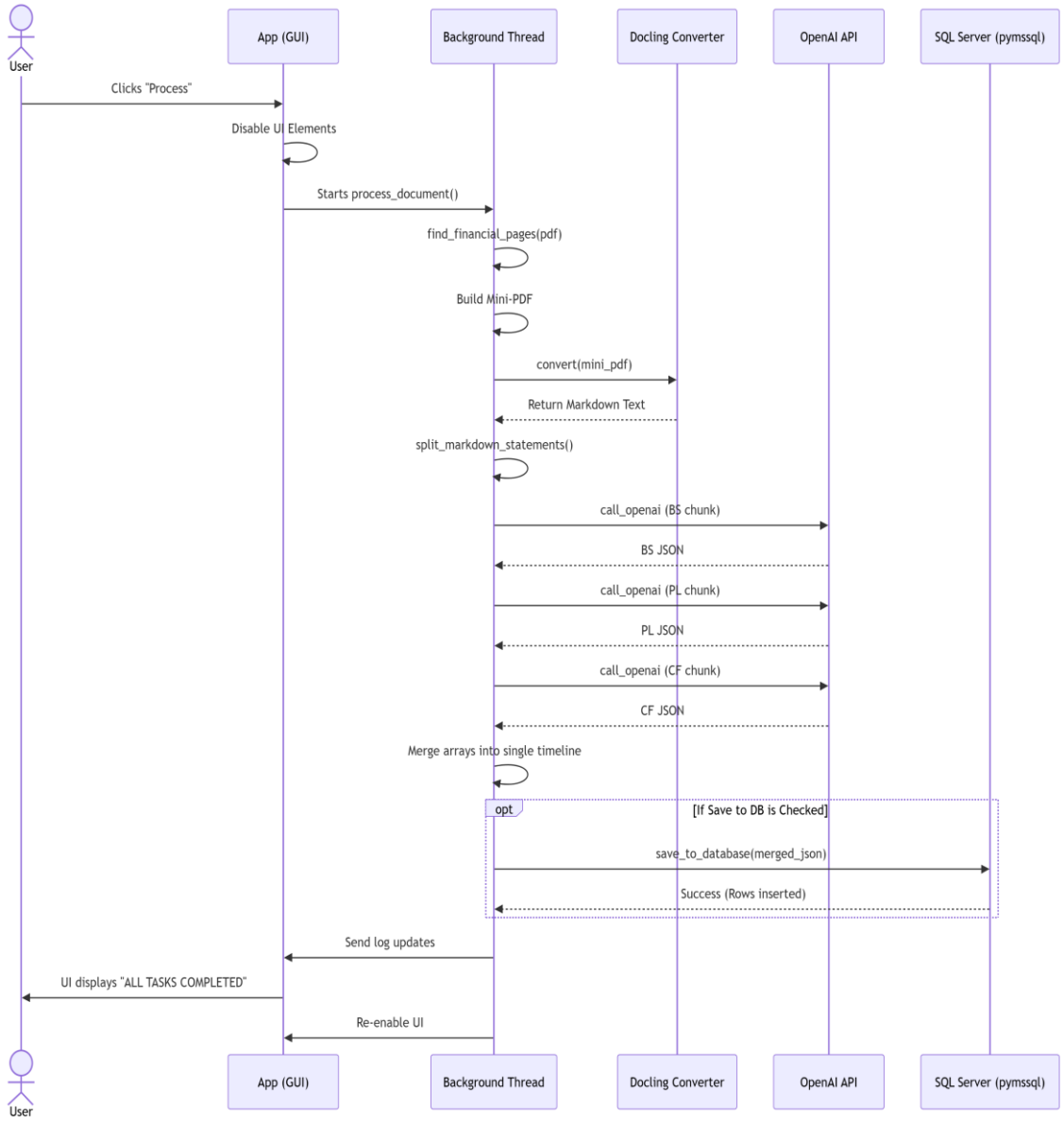


Figure 5: Interaction Diagram

Activity Diagram

❖ Activity Diagram of Blog

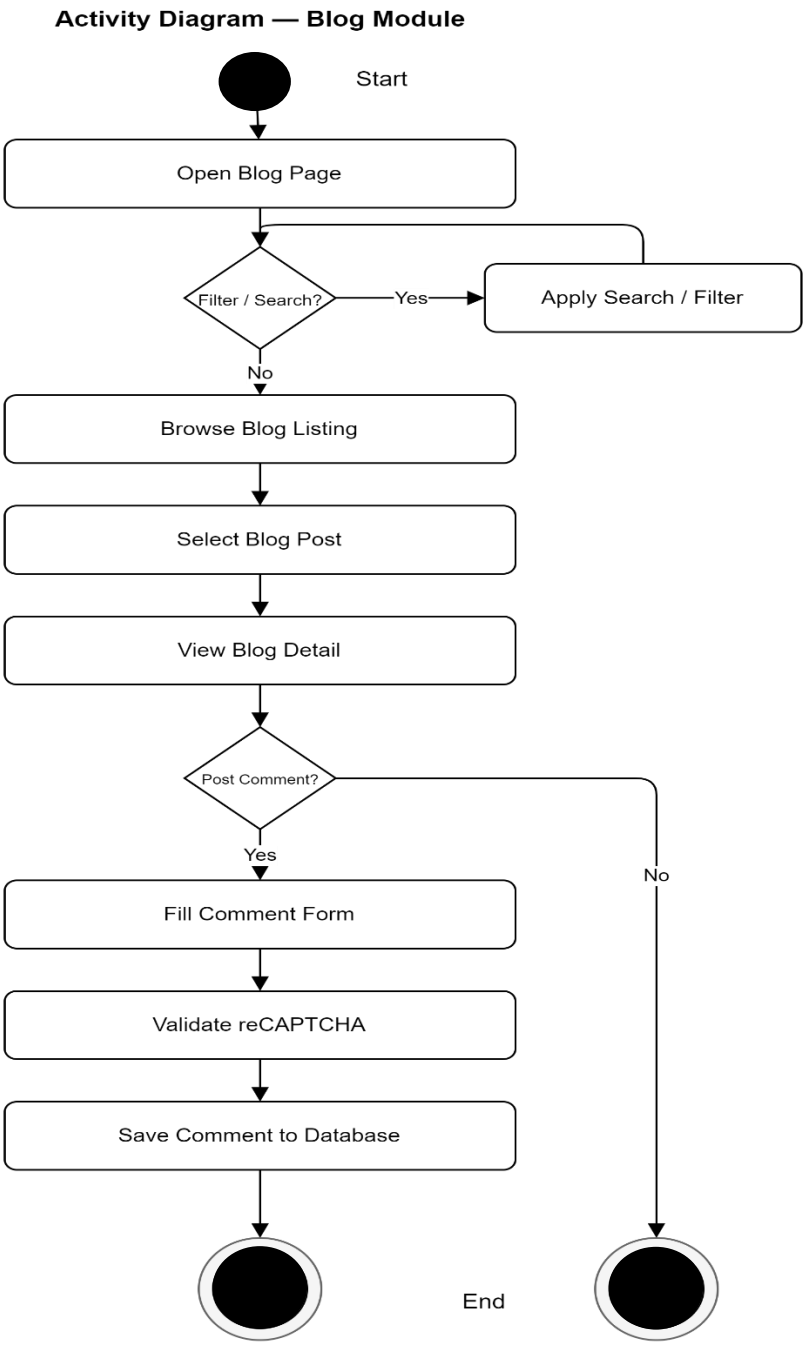


Figure 6: Activity Diagram of Blog

❖ Activity Diagram of Testimonial

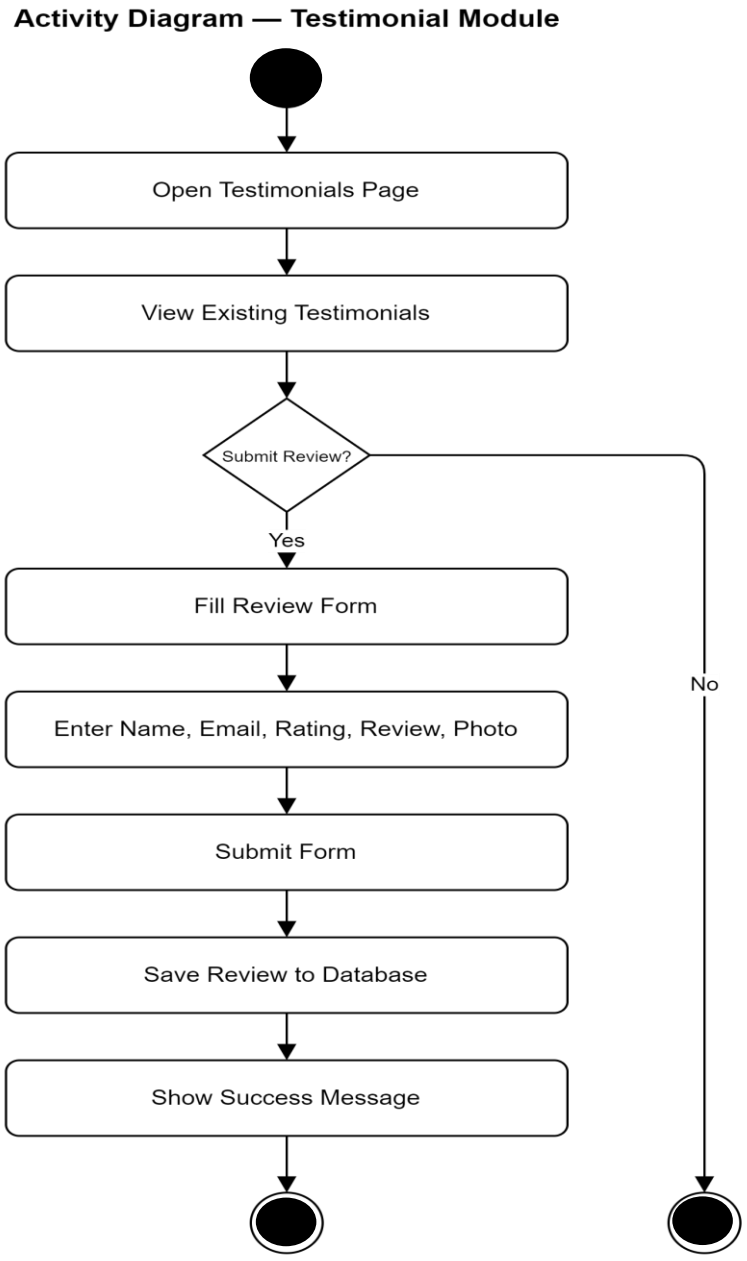


Figure 7: Activity Diagram of Testimonial

❖ **Activity Diagram of RHP Data Extraction**

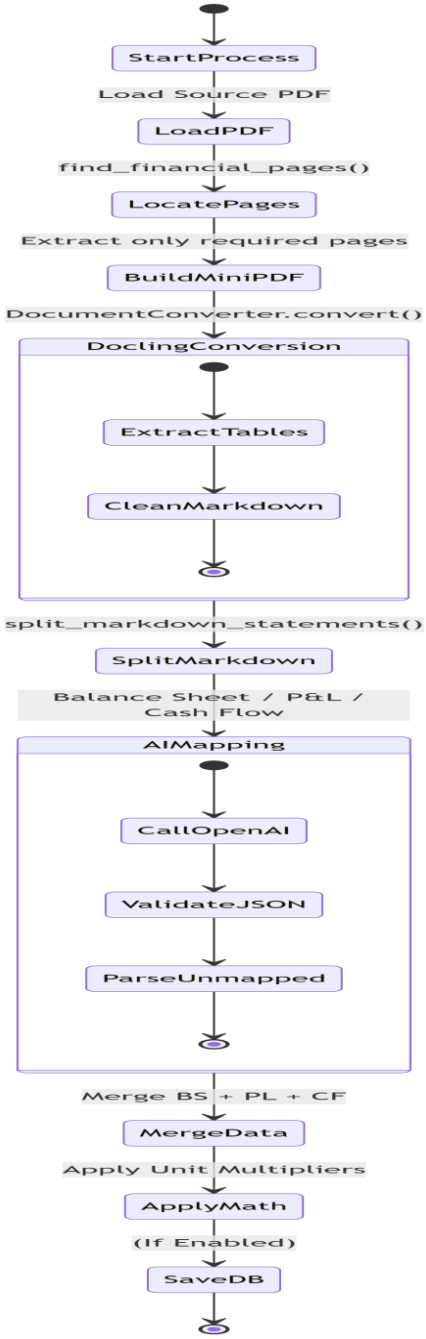


Figure 8: Activity Diagram of RHP Data Extraction

3.4 Data Dictionary

❖ Table Name: Blog

Name	Data Type	Constraint
id	Long	Primary Key
title	varchar(255)	Null
description	varchar(max)	Null
author_name	varchar(191)	Null
blog_category_id	Long	Foreign Key
link_name	varchar(191)	Null
seo_title	varchar(191)	Null
seo_description	varchar(255)	Null
seo_keyword	varchar(255)	Null
images	varchar(255)	Null
canonical_link	varchar(255)	Null
read_time	int	Null
is_active	bit (Boolean)	Null
created_on	datetime	Null

❖ Table Name: Testimonial

Name	Data Type	Constraint
id	int	Primary Key
name	varchar(191)	Null
mobile	varchar(20)	Null
email	varchar(191)	Null
profilePicImage	varchar(255)	Null
rating	int	Null
review	varchar(max)	Null
company	varchar(191)	Null
is_verified	bit (Boolean)	Null
is_active	bit (Boolean)	Null
is_for_home_page	bit (Boolean)	Null
created_on	datetime	Null

❖ Table Name: Users (Chatbot)

Name	Data Type	Constraint
UserID	int	Primary Key
Mobile	nvarchar(20)	Unique
FullName	nvarchar(100)	Null
Email	nvarchar(100)	Null
Step	nvarchar(50)	Null
CreatedAt	datetime	Null

❖ Table Name: UserQueries (Chatbot)

Name	Data Type	Constraint
QueryID	int	Primary Key
Mobile	nvarchar(20)	Foreign Key
QueryText	nvarchar(max)	Null
Status	nvarchar(50)	Null
SessionID	nvarchar(100)	Null
CreatedAt	datetime	Null

❖ Table Name: ChatLogs (Chatbot)

Name	Data Type	Constraint
LogID	int	Primary Key
Mobile	nvarchar(50)	Foreign Key
Sender	nvarchar(10)	Not Null
MessageText	nvarchar(max)	Not Null
CreatedAt	datetime	Null

❖ Table Name: tbl_corp_result (RHP Extraction Data)

Name	Data Type	Constraint
id	int	Primary Key
file_name	varchar(100)	Null
file_time	datetime	Null
ScripCode	varchar(20)	Null
NameOfTheCompany	varchar(500)	Null
DateOfBoardMeetingWhenFinancialResultsWereApproved	varchar(20)	Null
DateOfStartOfReportingPeriod	varchar(20)	Null
DateOfEndOfReportingPeriod	varchar(20)	Null
NatureOfReportStandaloneConsolidated	varchar(20)	Null
ReportingQuarter	varchar(20)	Null
DescriptionOfPresentationCurrency	varchar(10)	Null
LevelOfRoundingUsedInFinancialStatements	varchar(10)	Null
WhetherResultsAreAuditedOrUnaudited	varchar(20)	Null
IsCompanyReportingMultisegmentOrSingleSegment	varchar(20)	Null
DescriptionOfSingleSegment	varchar(1200)	Null
Symbol	varchar(100)	Null
MSEISymbol	varchar(200)	Null
ISIN	varchar(20)	Null
TypeOfCompany	varchar(10)	Null
DeclarationOfUnmodifiedOpinionOrStatementOnImpactOfAuditQualification	varchar(100)	Null
DateOfStartOfBoardMeeting	varchar(20)	Null
DateOfEndOfBoardMeeting	varchar(20)	Null
PropertyPlantAndEquipment	bigint	Null

4 Development

4.1 Coding Standards

To ensure code readability, maintainability, scalability, and consistency throughout the development process across the multi-language architecture of the TrueData Web Application, the following coding standards and best practices were strictly adopted:

1. Frontend (Razor, JavaScript/jQuery, HTML5/CSS3)

❖ **Project Structure**

- /Views (Organized by Controller names)
  /Views/Shared (For _Layout.cshtml and common partials)
  /Content (For custom CSS and Bootstrap files)
  /Scripts (For custom JS and jQuery libraries)

❖ **Code Style & Conventions**

- Follow camelCase for JavaScript variables and functions.
  Use semantic HTML5 tags for better SEO and accessibility.
  Avoid inline styles; strictly use external CSS files and Bootstrap 5 utility classes.

❖ **Best Practices**

- Centralize AJAX calls for dynamic content loading and form submissions to prevent page reloads.
  Validate all user inputs on the client side using jQuery Validation before submitting to the server.
  Keep Razor views clean; avoid writing complex C# business logic inside .cshtml files.

2. Backend Web & Middleware (C# ASP.NET MVC 5)

❖ **Project Structure**

- Organize clearly by application layers to enforce separation of concerns:
  /Controllers (HTTP request handlers)
  /Models (Data Transfer Objects and ViewModels)
  /Services (Business logic and validation)
  /TrueData.MarketAPI (Custom C# Class Library for external endpoints)

❖ **Code Style**

- Use PascalCase for Classes, Methods, and Properties (e.g., GetCorporateActions).
  Use camelCase for local variables and method parameters.
  Prefix private interface fields with an underscore (e.g., _dbContext).

❖ **Best Practices & Security**

- Maintain thin Controllers and push heavy business logic into the Services layer.
  Use asynchronous programming (async/await) to prevent blocking threads during external API calls.
  Implement Dual-Layer Security: Validate inputs using DataAnnotations and sanitize data against SQL Injection.
  Store sensitive configurations (API Keys, DB connections) securely in Web.config rather than hardcoding.

❖ **Error Handling**

- Implement centralized exception handling.
  Return proper HTTP status codes (400 for Bad Request, 500 for Internal Error).
  Use try-catch blocks effectively to log errors without exposing internal stack traces to the user.

3. Backend Data Services (Python, PdfPig, Pymssql)

❖ **Project Structure**

- Maintain a modular Python directory structured around the Extraction Pipeline:
  /extractors (PDF parsing logic)
  /mappers (Data normalization and unit conversions)
  /database (SQL insertion scripts)

❖ **Code Style**

- Strictly follow PEP 8 standards.
  Use snake_case for variables, functions, and modules (e.g., parse_rhp_table).
  Use PascalCase for Classes (e.g., PdfTableExtractor).

❖ **Best Practices**

- Isolate the execution environment using virtual environments (docling-env) to prevent dependency conflicts.
  Abstract mathematical unit conversions outside of the AI logic to ensure deterministic calculations.
  Handle file I/O operations safely using "with" context managers to prevent memory leaks during complex PDF parsing.

4. Database Management

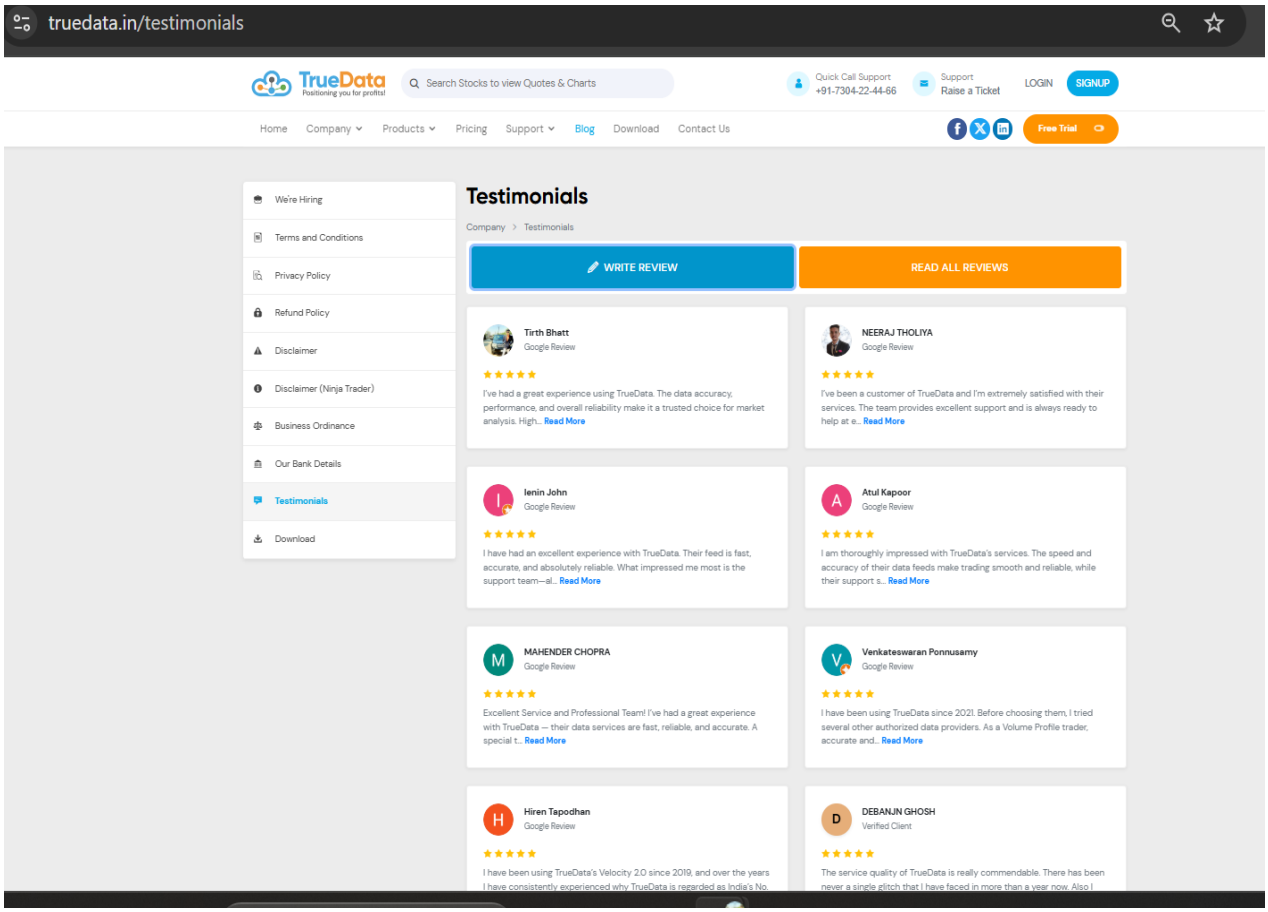
❖ **Conventions & Structure**

- Use clear, descriptive names for tables (e.g., tbl_corp_result, ChatLogs).
  Ensure every table has a defined Primary Key (Identity) and appropriate Foreign Keys for relational integrity.

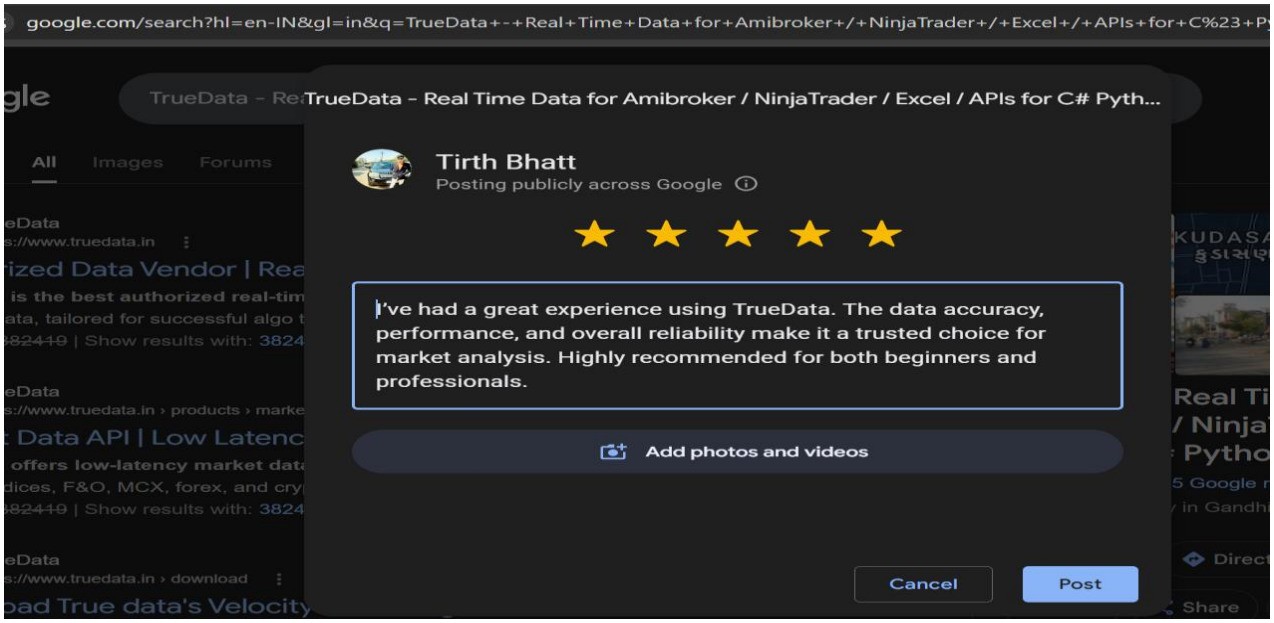
❖ **Best Practices**

- Strictly use Stored Procedures (SP) for all CRUD operations to optimize execution plans and prevent SQL injection.
  Connect the C# and Python backends to the database using secure micro-ORMs (Dapper/ADO.NET) and parameterized queries (pymssql).
  Index frequently queried columns (e.g., UserID, Mobile) to optimize read performance.

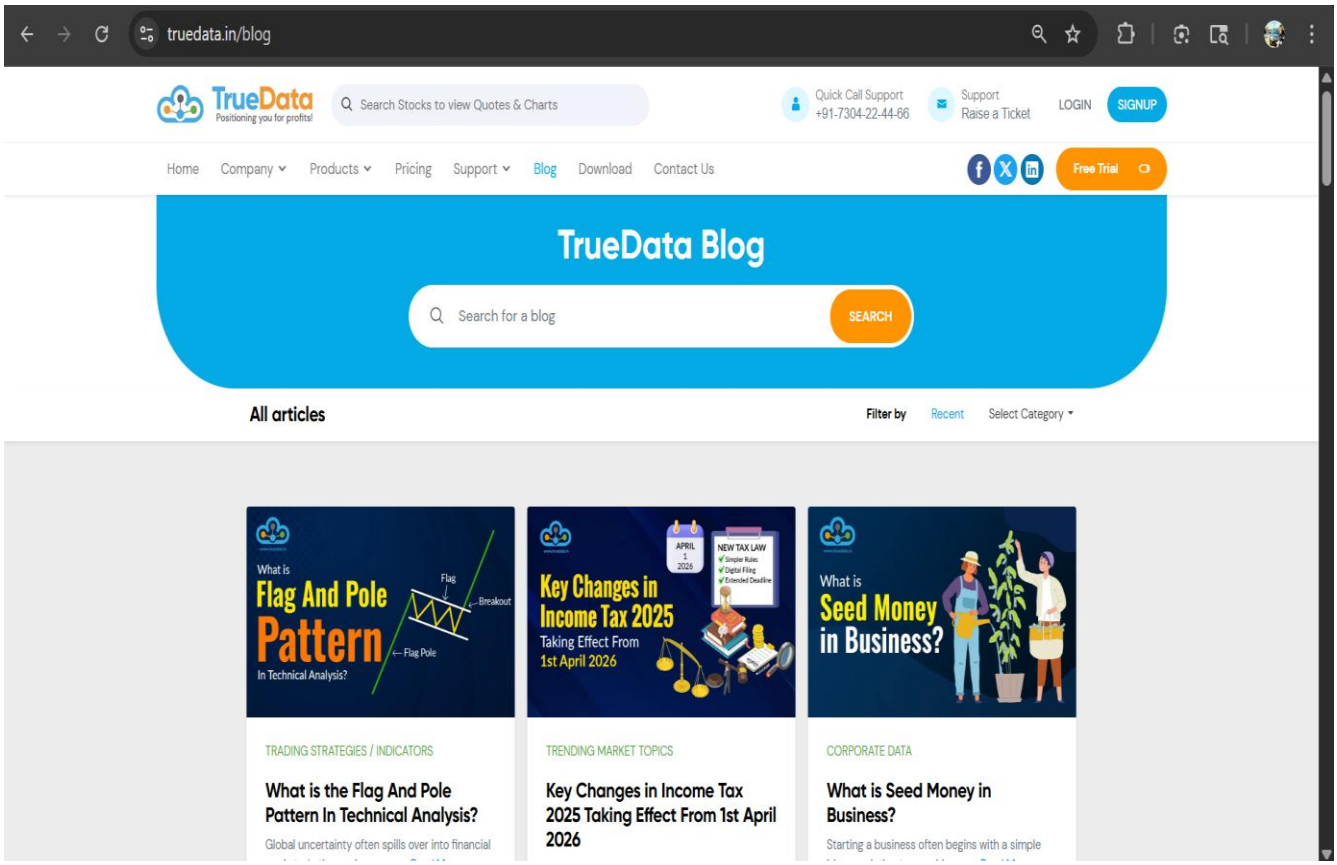
4.2 Screen Shots



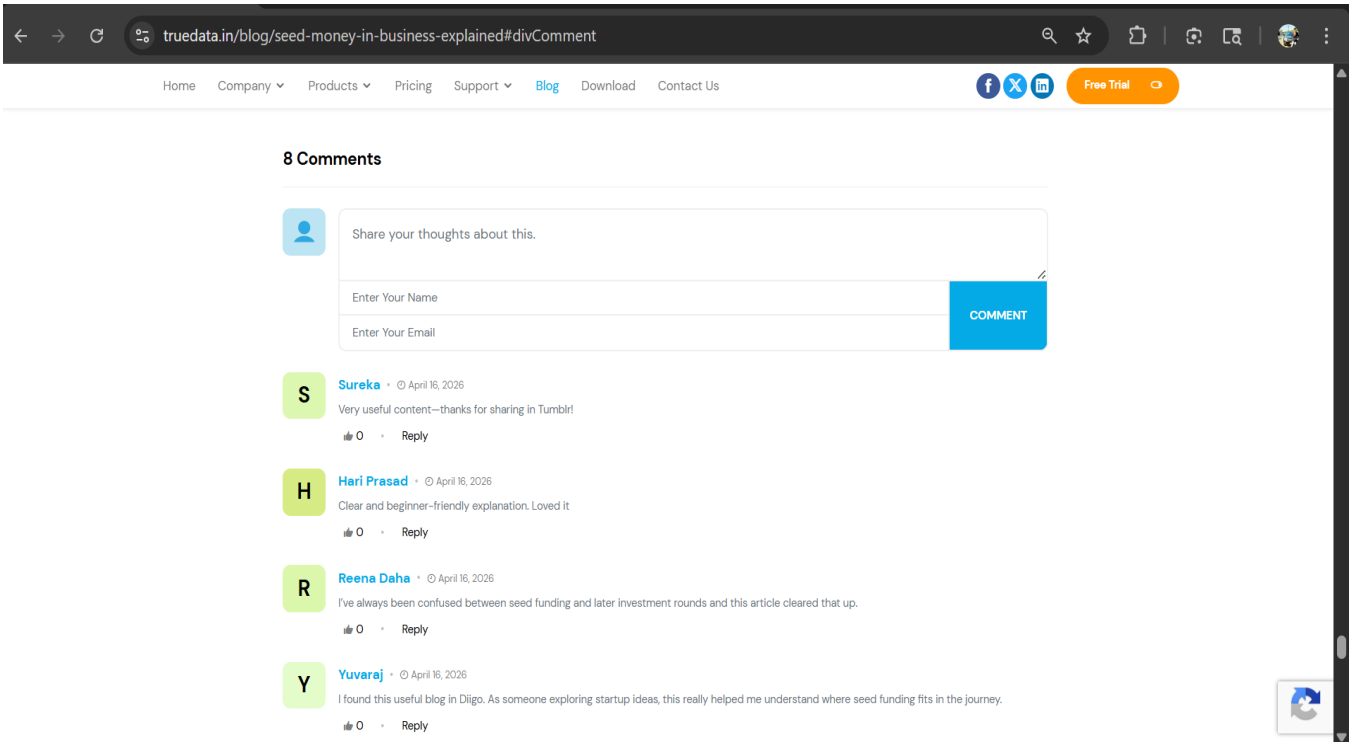
[Figure 1 :Truedata Testimonial page]



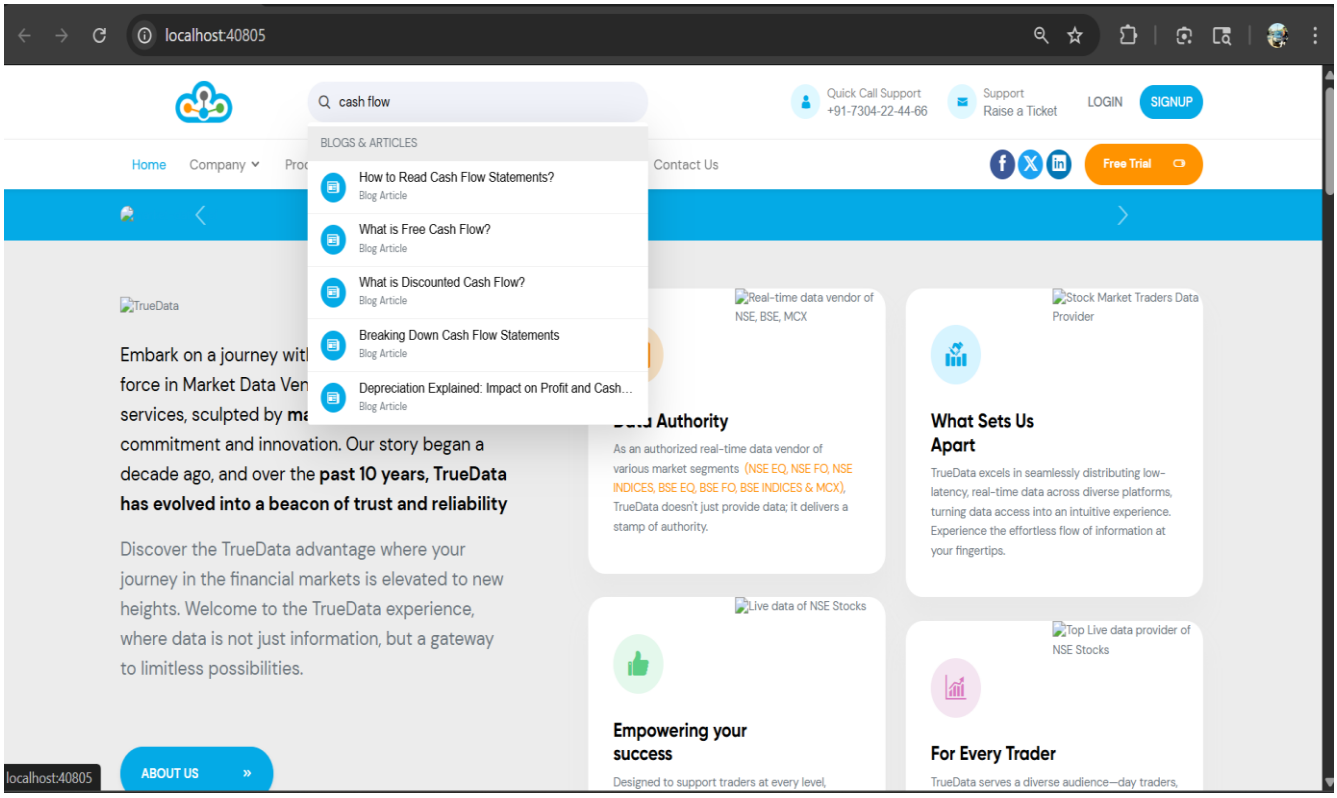
[Figure 2 :Review Submission]



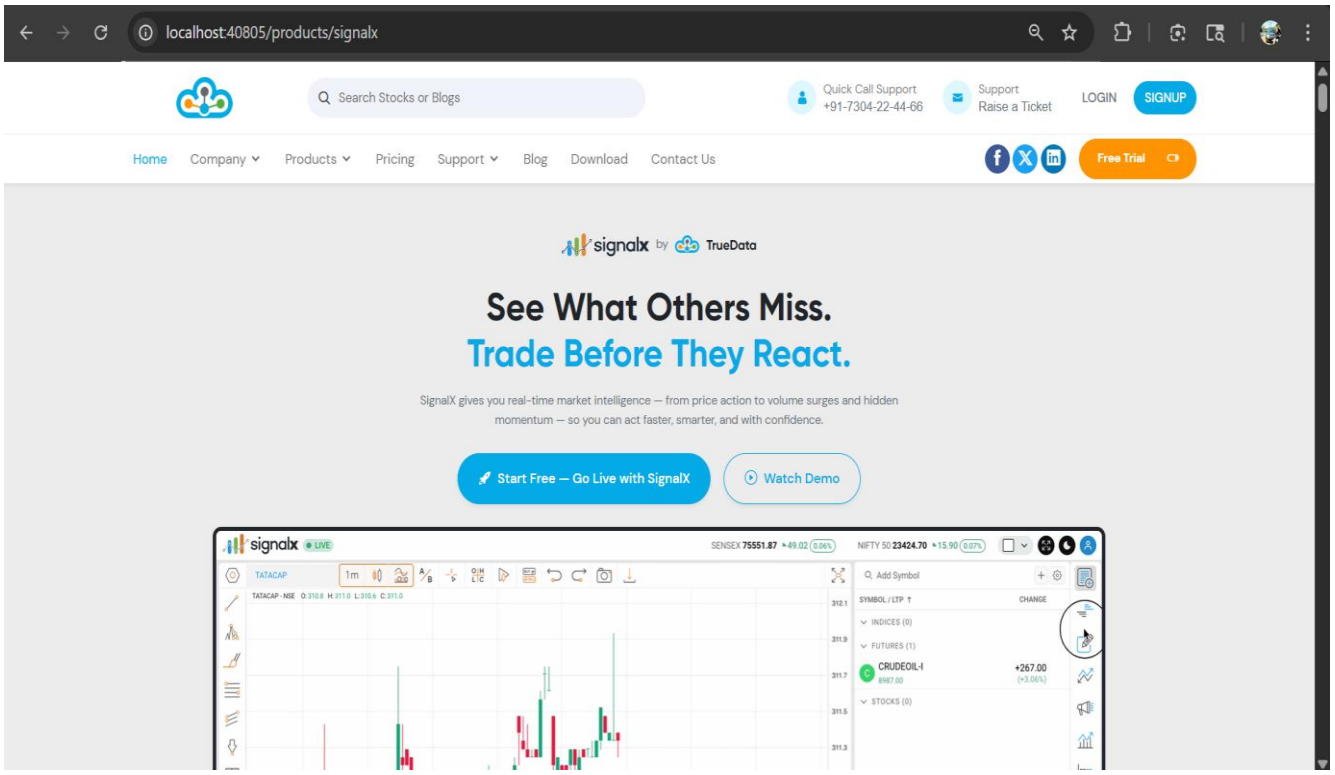
[Figure 3 :Truedata Blog page]



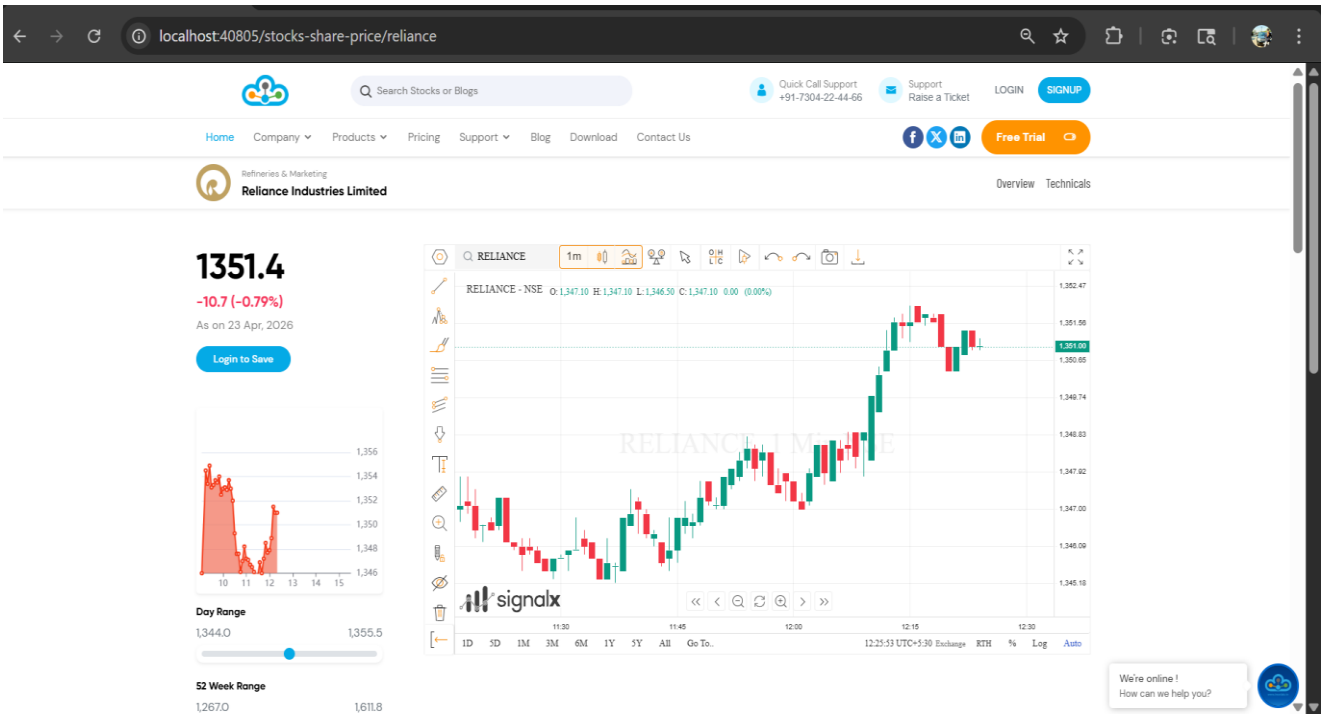
[Figure 4 :Truedata Blog Comment]



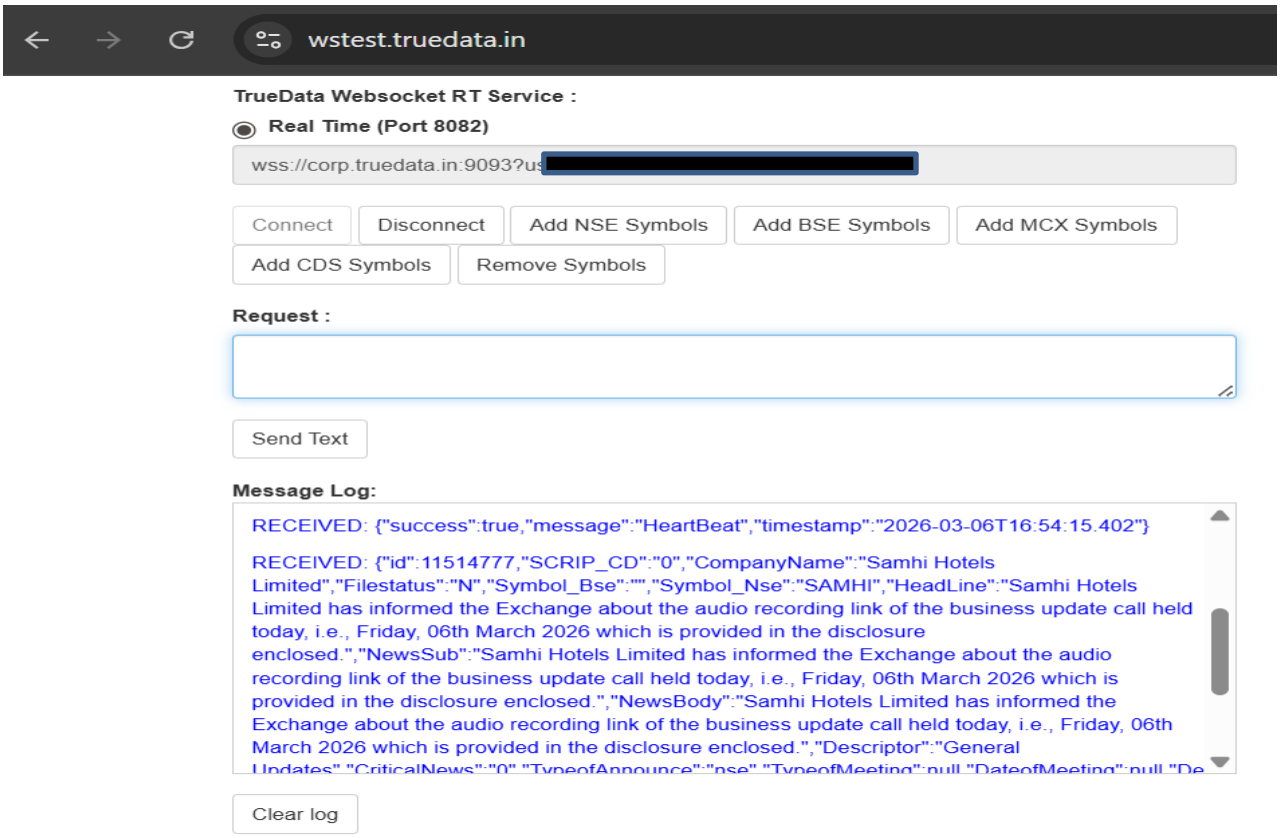
[Figure 5 :Global Search Functionality]



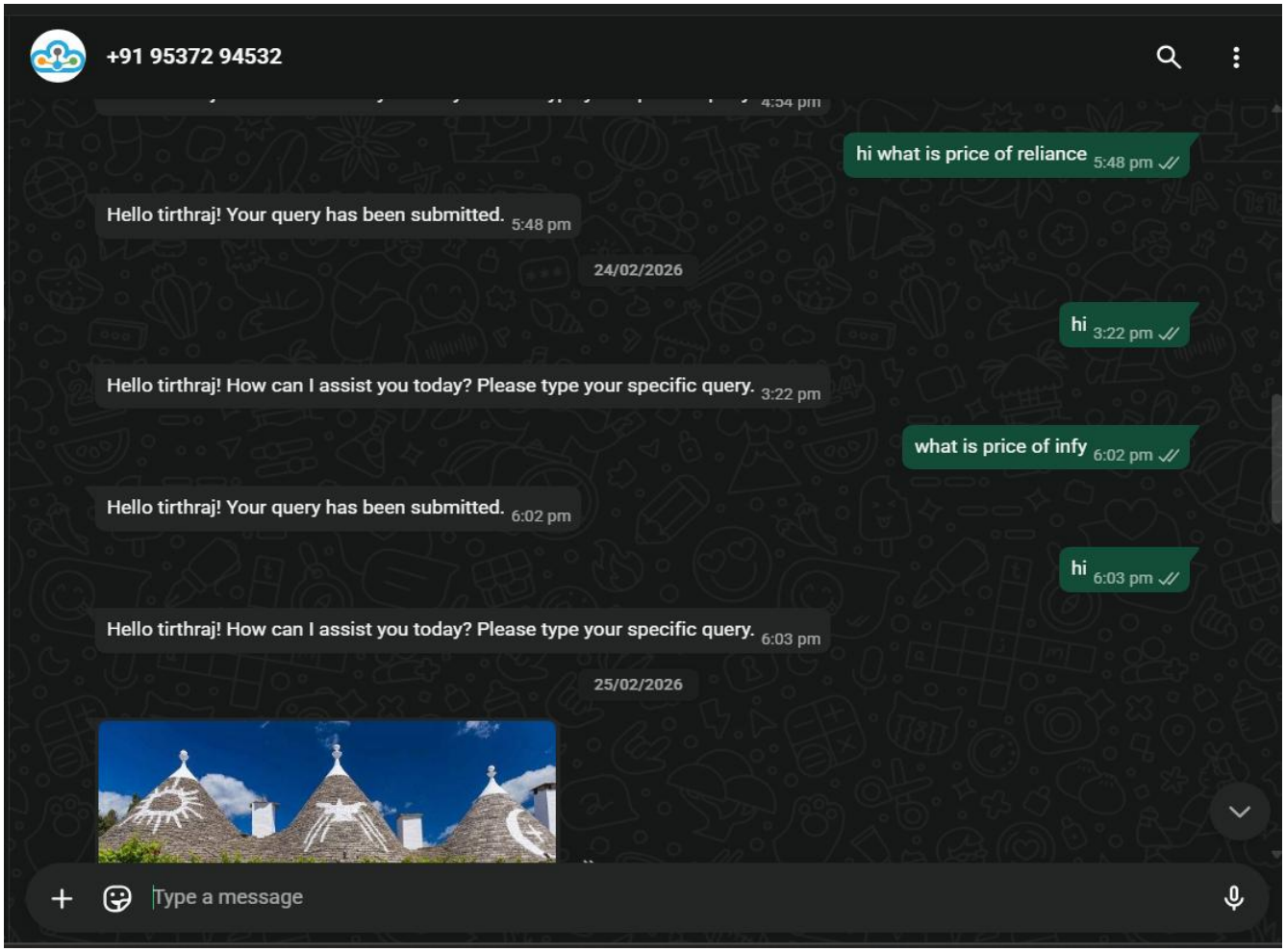
[Figure 6 :Truedata SignalX page]



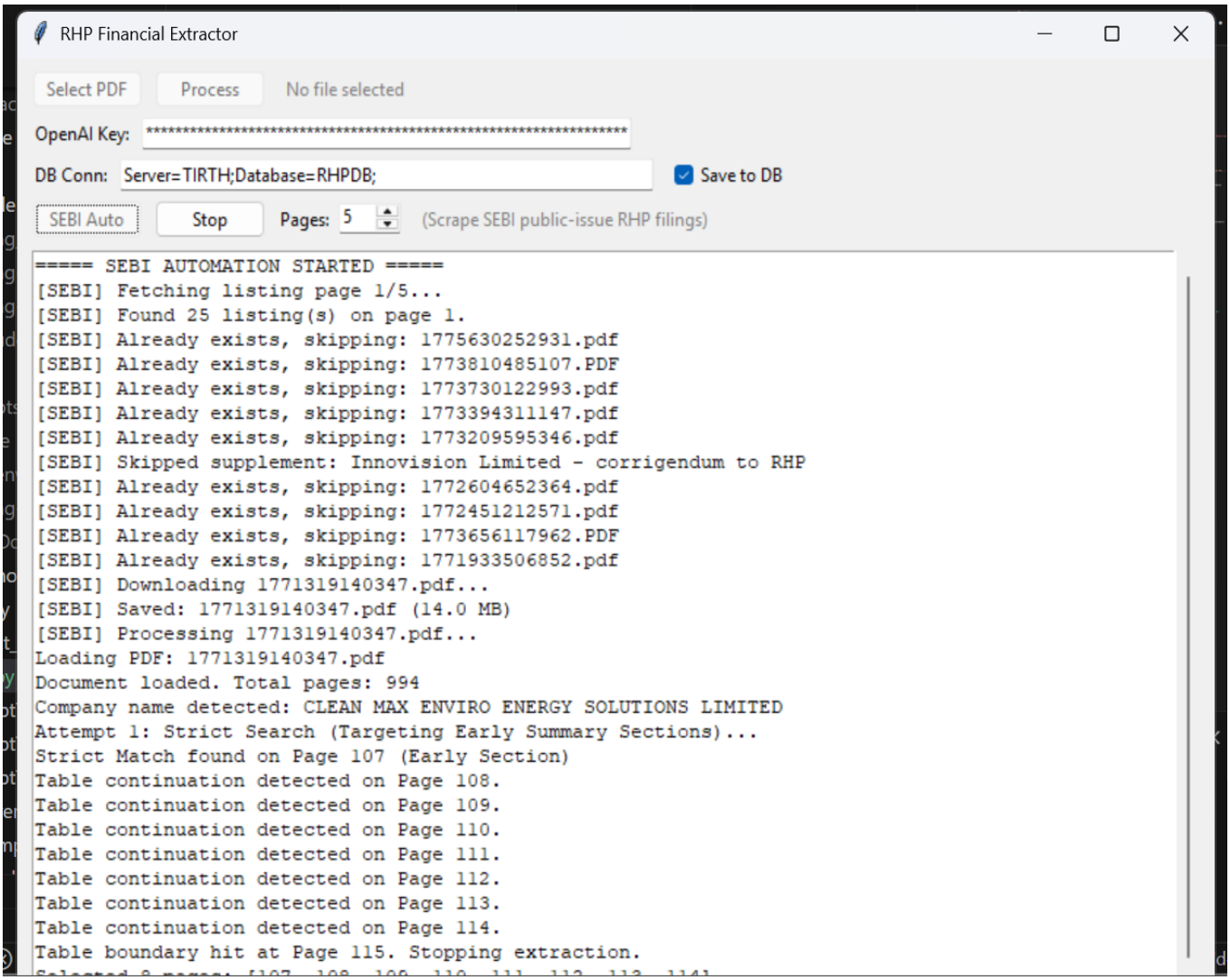
[Figure 7 :Truedata Charting page]



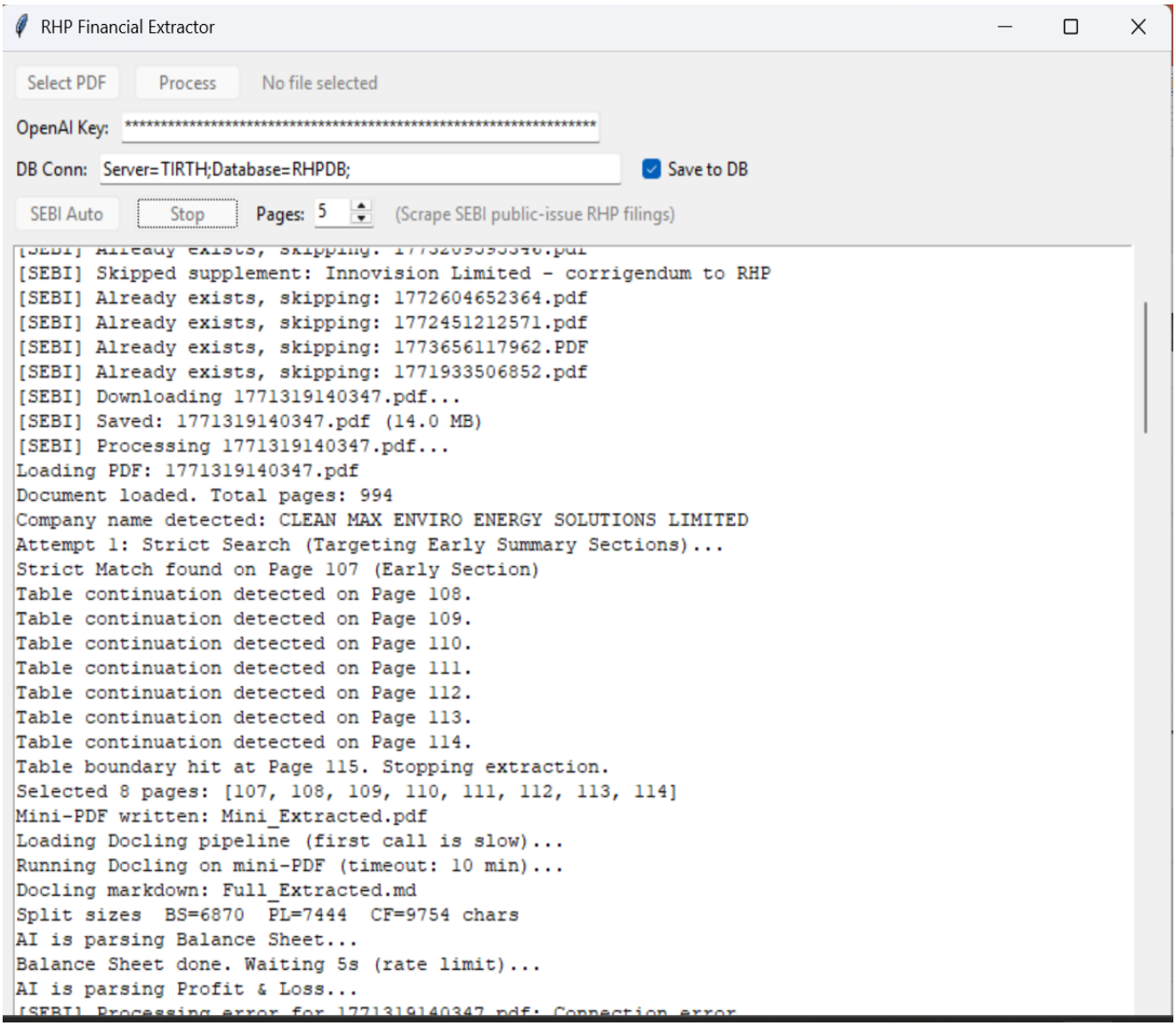
[Figure 8 :Truedata NSE Announcement page]



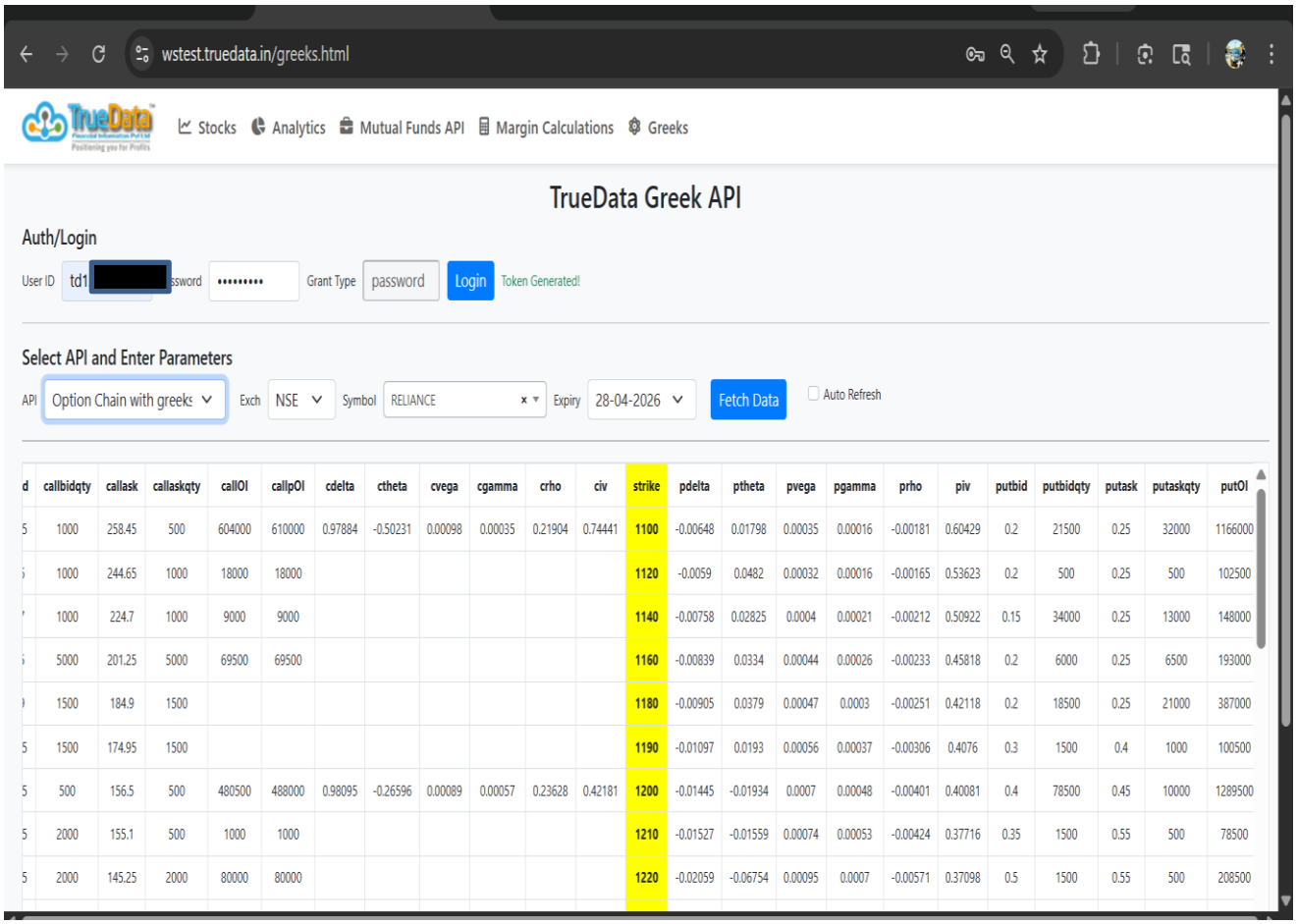
[Figure 9 :Whatsapp Chatbot]



[Figure 10 :RHP Data Extraction Process With Automation]

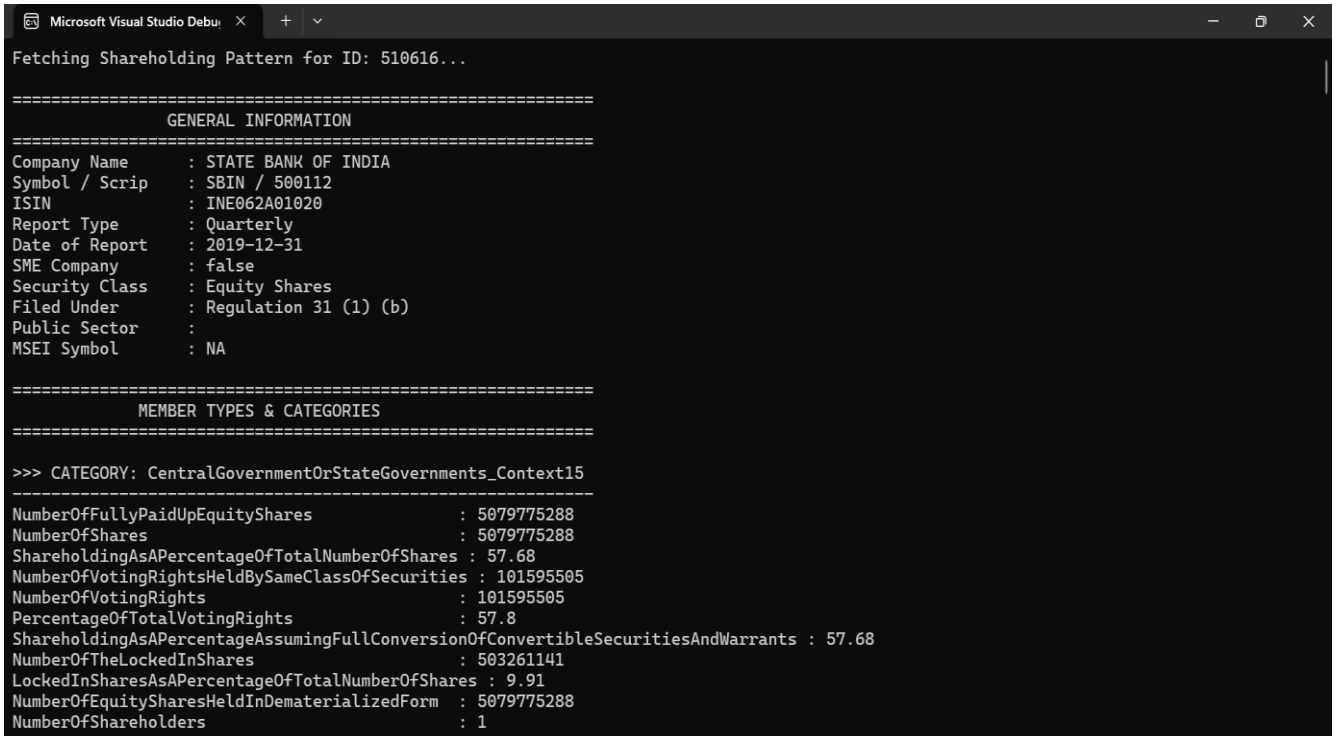


[Figure 11:Extraction of various pages]



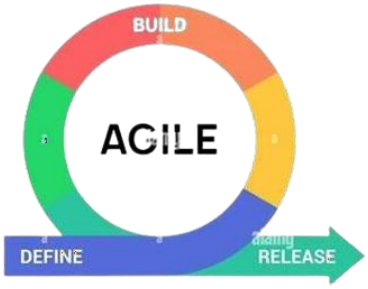
d	callbidqty	callask	callaskqty	callOI	callpOI	cdelta	ctheta	cvega	cgamma	crho	civ	strike	pdelta	ptheta	pvega	pgamma	prho	piv	putbid	putbidqty	putask	putaskqty	putOI
5	1000	258.45	500	604000	610000	0.97884	-0.50231	0.00098	0.00035	0.21904	0.74441	1100	-0.00648	0.01798	0.00035	0.00016	-0.00181	0.60429	0.2	21500	0.25	32000	1166000
5	1000	244.65	1000	18000	18000							1120	-0.0059	0.0482	0.00032	0.00016	-0.00165	0.53623	0.2	500	0.25	500	102500
7	1000	224.7	1000	9000	9000							1140	-0.00758	0.02825	0.0004	0.00021	-0.00212	0.50922	0.15	34000	0.25	13000	148000
5	5000	201.25	5000	69500	69500							1160	-0.00839	0.0334	0.00044	0.00026	-0.00233	0.45818	0.2	6000	0.25	6500	193000
9	1500	184.9	1500									1180	-0.00905	0.0379	0.00047	0.0003	-0.00251	0.42118	0.2	18500	0.25	21000	387000
5	1500	174.95	1500									1190	-0.01097	0.0193	0.00056	0.00037	-0.00306	0.4076	0.3	1500	0.4	1000	100500
5	500	156.5	500	480500	488000	0.98095	-0.26596	0.00089	0.00057	0.23628	0.42181	1200	-0.01445	-0.01934	0.0007	0.00048	-0.00401	0.40081	0.4	78500	0.45	10000	1289500
5	2000	155.1	500	1000	1000							1210	-0.01527	-0.01559	0.00074	0.00053	-0.00424	0.37716	0.35	1500	0.55	500	78500
5	2000	145.25	2000	80000	80000							1220	-0.02059	-0.06754	0.00095	0.0007	-0.00571	0.37098	0.5	1500	0.55	500	208500

[Figure 12 :Truedata Greek API Testing page]



[Figure 13 :C# Library for Corporate Data and Mutual Find Data]

5 Agile Documentation



5.1 Agile Project Charter

Vision: Build a secure, scalable financial data platform that automates complex workflows, provides starter blog interactions, verified testimonials, instant WhatsApp chatbot support, automated AI-driven RHP PDF extraction, and live market WebSocket integrations with strong SEO fundamentals.

GENERAL PROJECT INFORMATION	
Project Name:	TrueData Web Application
Project By:	Tirthraj
Start Date:	01-Jan-2026
End Date:	20-Apr-2026
Project Details:	To develop a comprehensive financial data analytics platform featuring starter blog comments, fundamental SEO optimizations, UI enhancements, an admin-verified testimonial module, an automated WhatsApp Chatbot, an AI-driven Python RHP extraction pipeline, and a dedicated Greeks API test page.

5.2 Agile Roadmap / Schedule

1st Phase (January)	2nd Phase (February)
• Understand existing system limitations • Finalize scope and architecture • Execute UI updates for key application pages • Implement fundamental SEO fixes (canonical tags, trailing slash issue, broken links)	• Develop starter blog comment section • Develop Testimonial Verification Module • Integrate Google reCAPTCHA v3 & Honeypot security • Setup WhatsApp Webhook integration • Implement Chatbot State-Machine Logic

3rd Phase (March)	4th Phase (April)
• Develop Python RHP Extractor (PdfPig) • Implement Data Mapping & Mathematical Unit Conversions • Build C# Middleware for Corporate Actions & Mutual Funds • Setup WebSocket Broadcasts for live NSE updates	• Develop Greeks API test functionality with multiple selected options • Full System Testing and Debugging • Performance Optimization (API and SQL SPs) • Final Documentation & Deployment

5.3 Agile Project Plan

Task Name	Responsible	Start Date	End Date	Days	Status
Sprint 1 (01/01/2026 - 21/01/2026)					
Requirements & System Architecture	Tirthraj	01/01/2026	06/01/2026	6	Completed
UI Updates for Key Pages	Tirthraj	07/01/2026	14/01/2026	8	Completed
SEO Fixes (Canonical, Trailing Slash, Broken Links)	Tirthraj	15/01/2026	21/01/2026	7	Completed
Sprint 2 (22/01/2026 - 12/02/2026)					
Starter Blog Comment Section	Tirthraj	22/01/2026	26/01/2026	5	Completed
Testimonial Module & Admin Gate	Tirthraj	27/01/2026	03/02/2026	8	Completed
reCAPTCHA & Honeypot Integration	Tirthraj	04/02/2026	08/02/2026	5	Completed
WhatsApp Webhook Configuration	Tirthraj	09/02/2026	12/02/2026	4	Completed
Sprint 3 (13/02/2026 - 05/03/2026)					
Chatbot State Logic & Query Logging	Tirthraj	13/02/2026	20/02/2026	8	Completed

Python RHP Extractor (PdfPig)	Tirthraj	21/02/2026	28/02/2026	8	Completed
RHP Unit Conversion & SQL Mapping	Tirthraj	01/03/2026	05/03/2026	5	Completed
Sprint 4 (06/03/2026 - 26/03/2026)					
C# Middleware (Corp & Mutual Funds)	Tirthraj	06/03/2026	13/03/2026	8	Completed
Live WebSocket NSE Announcements	Tirthraj	14/03/2026	20/03/2026	7	Completed
Greeks API Testing Functionality	Tirthraj	21/03/2026	26/03/2026	6	Completed
Sprint 5 (27/03/2026 - 20/04/2026)					
Full System QA (Anti-Spam & Chatbot)	Tirthraj	27/03/2026	05/04/2026	10	Completed
API Optimization & Load Testing	Tirthraj	06/04/2026	12/04/2026	7	Completed
Final Documentation & Presentation	Tirthraj	13/04/2026	20/04/2026	8	Completed

5.4 Agile User Stories (Minimum 3 Tasks)

Automated Data Processing	
Title:	Extract Financial Data from RHP PDFs
User Story	As an Administrator, I want to upload complex RHP PDF files so that the AI pipeline can automatically extract and structure the financial tables into the database.
Acceptance Criteria	Given I am logged into the Admin panel,

	When I upload a valid RHP PDF document, Then the Python PdfPig module should parse the tables, automatically execute mathematical unit conversions (e.g., Millions to Crores), and save the structured data into the SQL database. Tasks: • Develop PdfPig parsing logic. • Implement dynamic unit conversion logic. • Create pymssql database insertion scripts.

Real-Time Interactions	
Title:	Automated WhatsApp Query Resolution
User Story	As a Registered User, I want to interact with a WhatsApp chatbot so that I can receive instant, 24/7 support for my financial queries.
Acceptance Criteria	Given I have registered my mobile number, When I send a message to the TrueData WhatsApp number, Then the chatbot should process my query, track the conversation state, and reply automatically while logging the session. Tasks: • Integrate WhatsApp Webhook. • Develop state-machine logic ('NEW', 'PENDING'). • Create ChatLogs and UserQueries database structures.

API Testing & Discovery	
Title:	Verify Greeks API Functionality
User Story	As a Financial Analyst, I want to use a functional testing page with multiple selected options so that I can easily check and verify the data of the Greeks API.
Acceptance Criteria	Given I am on the Greeks API test page, When I select multiple options and submit the form, Then I should see the accurate, real-time Greeks data fetched and displayed correctly. Tasks: • Design testing UI with multiple option selectors. • Integrate C# middleware to call the Greeks API. • Parse and format the returned JSON response.

5.5 Agile Release Plan

Task Name	Start Date	End Date	Status
Sprint 1: UI Updates & SEO Fixes	01/01/2026	21/01/2026	Released
Sprint 2: Blog Comments & Testimonials	22/01/2026	12/02/2026	Released
Sprint 3: Chatbot & Python Extractor	13/02/2026	05/03/2026	Released
Sprint 4: Middleware & Greeks Testing	06/03/2026	26/03/2026	Released
Sprint 5: Testing, Optimization & Deploy	27/03/2026	20/04/2026	Released

5.6 Agile Sprint Backlog

Task Name	Story	Sprint Ready	Priority	Status	Story Point	Assigned Sprint
UI Updates for Key Pages	Yes	Yes	High	Done	5	Sprint 1
SEO: Canonical & Trailing Slash Fixes	Yes	Yes	High	Done	8	Sprint 1
Starter Blog Comments	Yes	Yes	Medium	Done	5	Sprint 2
Testimonial Moderation	Yes	Yes	Medium	Done	5	Sprint 2
Chatbot State Logic	Yes	Yes	High	Done	8	Sprint 3
PdfPig Data Extraction	Yes	Yes	High	Done	13	Sprint 3
C# API Middleware	Yes	Yes	High	Done	8	Sprint 4
Greeks API Testing Options	Yes	Yes	Medium	Done	5	Sprint 4

5.7 Agile Test Plan

Date	Test	Description	Expected Result	Actual Result	Passed	Tested By
20/01/2026	1	SEO Trailing Slash & Canonical	System resolves trailing slashes and outputs canonical tags correctly	As Expected	Passed	Tirthraj
02/02/2026	2	Honeypot Trap Test	Bot submission silently discarded	As Expected	Passed	Tirthraj
18/02/2026	3	Chatbot DB Logging	WhatsApp states map to UserQueries	As Expected	Passed	Tirthraj
04/03/2026	4	RHP Unit Conversion	Millions convert to Crores correctly	As Expected	Passed	Tirthraj
18/03/2026	5	WebSocket Broadcasts	Zero-latency NSE updates fire	As Expected	Passed	Tirthraj
25/03/2026	6	Greeks API Multiple Select	API handles multi-option payload	As Expected	Passed	Tirthraj
10/04/2026	7	End-to-End System Load	Platform remains stable under load	As Expected	Passed	Tirthraj

5.8 Earned-Value and Burn Charts

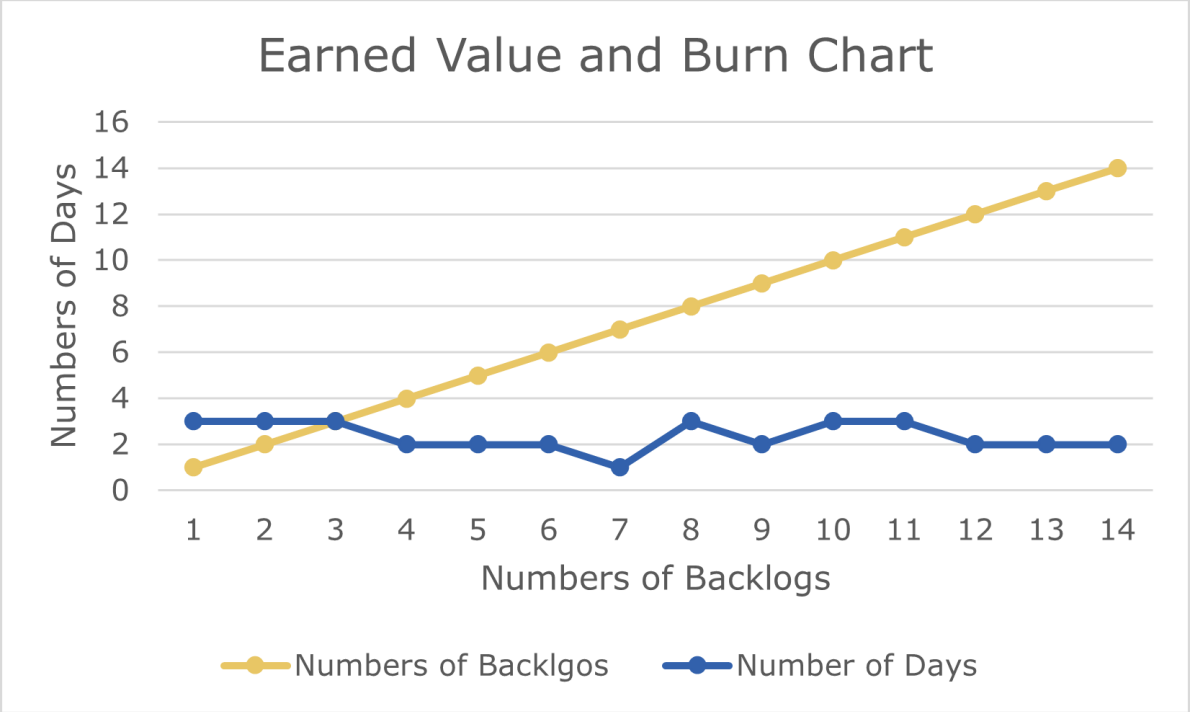


Figure 49: Earned-Value and Burn Chart

6. Proposed Enhancements

While the TrueData Web Application successfully fulfills its core objectives of automating financial data extraction, providing SEO-optimized CMS functionalities, and delivering real-time chatbot support, several future enhancements can be introduced to improve its efficiency, scalability, and user experience:

Advanced NLP Chatbot Integration

o Upgrade the current state-machine WhatsApp chatbot to utilize advanced Natural Language Processing (NLP) models (e.g., Dialogflow or OpenAI integration) to handle highly complex, unscripted financial queries.

o Implement sentiment analysis to escalate frustrated users to human administrators automatically.

Expanded OCR & AI Document Parsing

o Enhance the Python PdfPig module to support Optical Character Recognition (OCR), allowing the system to extract data from scanned, non-text-searchable Red Herring Prospectuses (RHPs).

o Expand AI mapping rules to automatically parse and structure Annual Reports and Quarterly Earnings Transcripts beyond standard RHPs.

Cross-Platform Mobile Application

o Develop a dedicated React Native or Flutter mobile application (Android & iOS) to provide administrators and analysts easier access to the dashboard and extracted financial tables on the go.

Advanced Dashboard & Data Visualization

o Integrate dynamic charting libraries (e.g., Chart.js or D3.js) to visually represent the extracted Balance Sheet and P&L data.

o Provide downloadable PDF/Excel reports comparing financial metrics across different companies automatically.

Multi-Language Localization

o Implement full website localization utilizing .resx files to support multiple regional Indian languages (Hindi, Gujarati) for broader accessibility.

Enhanced Security & Role Management

o Introduce Two-Factor Authentication (2FA) for administrative logins to further secure the CMS and extraction modules.

o Implement granular, customizable Role-Based Access Control (RBAC) matrices allowing specific analysts to view only certain categories of extracted financial data.

7. Conclusion

- The TrueData Web Application successfully demonstrates how technology can simplify and enhance the management and analysis of financial data through a structured, highly secure, and automated platform. By integrating modules such as an SEO-optimized blog CMS, verified testimonials, a real-time WhatsApp chatbot, and automated Python-based RHP PDF extraction, the system addresses key challenges faced by both end-users and financial administrators.

- Through this project, the primary objectives of improving digital footprint via SEO, providing instant automated client support, eliminating manual data entry errors, and ensuring robust anti-spam security (Honeypot & reCAPTCHA) were successfully achieved. The platform benefits users by providing quick resolutions via WhatsApp and provides analysts with high-fidelity, cleanly structured financial data extracted directly from complex prospectuses.

- Although certain limitations remain, such as the system's dependency on standard document formatting for PDF extraction and reliance on external API rate limits, the project provides a strong foundation for future scalability. Features such as advanced NLP chatbot integration, OCR support for scanned documents, and data visualization dashboards can significantly extend the system's capabilities.

- In conclusion, the TrueData Web Application meets its intended objectives and proves to be a highly practical, secure, and modern solution for financial data operations. With continuous improvements, it has the potential to evolve into a comprehensive, fully automated financial intelligence and client management platform.

8. Bibliography

During the development of the TrueData Web Application, reference was taken from various official documentations, video tutorials, and technical forums:

• Microsoft ASP.NET MVC 5 Documentation: https://learn.microsoft.com/en-us/aspnet/mvc/

• Microsoft C# Programming Guide: https://learn.microsoft.com/en-us/dotnet/csharp/

• Microsoft SQL Server Documentation: https://learn.microsoft.com/en-us/sql/sql-server/

• PdfPig (Python PDF parsing library) Documentation: https://pdfpig.readthedocs.io/

• Pymssql (Python SQL Server driver) Documentation: https://pymssql.readthedocs.io/

• Google reCAPTCHA v3 Developer Guide: https://developers.google.com/recaptcha/docs/v3

• Meta WhatsApp Business API Documentation: https://developers.facebook.com/docs/whatsapp

• Bootstrap 5 Official Documentation: https://getbootstrap.com/docs/5.0/getting-started/introduction/

• jQuery API Documentation: https://api.jquery.com/

• Postman API Platform: https://www.postman.com/

• Stack Overflow Community: https://stackoverflow.com